

UNIVERSITY OF
CAMBRIDGE
Department of Computer
Science and Technology

May 15, 2026

Efficient Coinductives through State-Machine Corecursors

Submitted in partial fulfilment of the requirements for the
Computer Science Tripos, Part II 2026

Declaration

I, the candidate for Part II of the Computer Science Tripos with Blind Grading Number 5065D, hereby declare that this report and the work described in it are my own work, unaided except as may be specified below, and that the report does not contain material that has already been used to any substantial extent for a comparable purpose. In preparation of this report, I adhered to the Department of Computer Science and Technology AI Policy. I am content for my report to be made available to the students and staff of the University.

Date: May 15, 2026

Proforma

BGN	5065D		
Title	Efficient Coinductives through State-Machine Corecursors		
Word count	9911	**Line count**	11857
Examination	CST Part II	**Year**	2026
Day-to-day supervisor	Alex Keizer Tobias Grosser	**Marking supervisor**	Jamie Vicary
Project originator	The candidate		
Ethics approval	N/A		

Project Aims

Create the first high performance coinduction library outside of the kernel. Construct simple coinductive types in this system.

Work completed

Implemented the first constant time coinduction library. Constructed rich data structure for program verification and denotational semantics. Improved ergonomics for coinductive function writing.

Acknowledgements

Alex Keizer For supervising this dissertation and giving me the opportunity to explore coinductive types.
Tobias Grosser For taking over supervision when Alex could not and pushing me to go further.
Jamie Vicary For marking this project on short notice and providing insightful feedback.
Nicolas Chappe For taking a genuine interest in the project and helping me with ITrees.
Wojciech Różowski For helping with project specifics and technical parts of coinductives.
Russell Moore For reviewing my dissertation and help me identify specialist language.

Contents

1 Introduction 1
2 Preparation 2
2.1 Dependent type theory 2
2.2 Universe levels 2
2.3 Lean 3
2.3.1 Mathlib 4
2.3.2 The tactic language 4
2.3.3 The Lean compiler 4
2.4 Polynomial functors 5
2.4.1 Common polynomials 6
2.5 Recursion and Inductive data type 7
2.6 Corecursion and coinductive types 7
2.6.1 In OCaml 7
2.6.2 Streams 8
2.6.3 The M-type 8
2.7 Bisimilarity 10
2.8 The Free Monad 10
2.9 Interaction Trees 11
2.10 Methodology 11
2.11 Starting point 12
2.12 Tools used 12
2.13 Requirement Analysis 13
2.13.1 State-Machine encoding (Core) ... 13
2.13.2 The delay monad (Core) 13
2.13.3 AltRepr Type (Extension) 13
2.13.4 Interaction Trees (Extension) 13
2.13.5 Free Monad (Extension) 14
3 Implementation 15
3.1 Stream implementation 15
3.1.1 State-machine streams 15
3.1.2 Progressive approximation streams 16
3.1.3 State-machine Progressive approximation equivalence 17
3.2 State-machine encoding of M-types 18
3.2.1 Rephrasing bisimilarity for M-types 18
3.2.2 PreM 18
3.2.3 State-machine M-type (SM) 19
3.2.4 State-machine Progressive approximation equivalence 20
3.3 The Alternative Representation (AltRepr) Type 21
3.3.1 The $\mathcal{U}$-universe State-Machine M-type 22
3.4 The delay monad 23
3.5 Interaction Trees 23
3.5.1 The ITree Monad 25
3.5.2 Weak bisimilarity 25
3.5.3 A formally verified optimization . 26
3.6 Free Monad 27
3.7 Machinery 29
3.7.1 Transliteration 29
3.7.2 Expanding the progressive approximation theory 29
3.7.3 Polynomial equivalents 30
3.7.4 Natural isomorphisms of MvFunctors 30
4 Evaluation 32
4.1 State-Machine encoding (core) 32
4.2 AltRepr Type (extension) 34
4.3 Interaction Trees (extension) 34
4.4 The delay monad (core) 35
4.5 The Free monad (extension) 35
5 Conclusions 37
5.1 Future work 37
5.1.1 Syntax macro for types 37
5.1.2 Derecursifier for corecursive function 37
Bibliography 38
6 Appendicies 40
6.A Inductive predicate 40
6.B Coinductive predicate 40
6.C Benchmarking code 40
6.D Unsoundness of old shrink definition 43
6.E Project import graph 44
7 Proposal 46

List of figures

Figure 1 Inference rules for Π and Σ types 2
Figure 2 Curry-Howard correspondence 2
Figure 3 Lean compiler layout 3
Figure 4 Some strong-induction primitives from Lean 5
Figure 5 Gcd as Source, Kernel and LCNF 5
Figure 6 Formal definition of polynomial functor 6
Figure 7 Projection polynomial 6
Figure 8 Constant polynomial 6
Figure 9 Product polynomial 6
Figure 10 Sum polynomial 6
Listing 1 Definition of list in OCaml 7
Listing 2 Definition of colist in OCaml 8
Figure 11 Agreement of two trees of height 1 and 2 9
Listing 3 Definition of approx for stream 9
Listing 4 Objects of the corecursor for streams 9
Listing 5 Destructor for streams 9
Figure 12 Two bisimilar state-machines 10
Listing 6 Bisimilarity on streams 10
Figure 13 Choice points for a producing a stream 11
Listing 7 Notational defintion for ITrees 11
Figure 14 Birds-eye view of components of the project 15
Listing 8 PreStream and destructors 16
Figure 15 `PreStream` setoid 16
Figure 16 `corec` and destructors for SStreams 16
Figure 17 Coinduction principle for SStreams 16
Figure 18 PStream definition 17
Figure 19 PStream destructors 17
Figure 20 Coinduction principle for PStreams 17
Listing 9 `corec` for PStreams 17
Listing 10 Equivalence between progressive-approximation and state-machine streams 17
Figure 21 Equivalent bisimilarity definitions 18
Listing 11 `PreM` definition 18
Figure 22 Destructor and β-rule for `PreM` 19
Listing 12 ULifting of `PreM` 19
Listing 13 Bisimilarity of `PreM` types 19
Figure 23 Equivalence relation for `PreM` bisimilarity. 19
Listing 14 Bisimilarity of `SM`-types 20
Listing 15 Functions of the equivalence 20
Listing 16 Equivalence between state-machine and progressive approximation encoded M-types `equivP` 21
Listing 17 Mathlib definition of `Shrink` 21
Figure 24 Universe transforming types. 22
Figure 25 Defintions of ITrees 23
Table 1 Functions implemented in the dissertaton[L], in Rocq[R] and M. Sammler[M] 24
Table 2 Equations implemented in the dissertaton[L], in Rocq[R] and M. Sammler[M] 24
Figure 26 Visual representation of `bind` on ITrees 25

Figure 27 Weak-bisimilarity for ITrees 26
Listing 18 `Imp` and `simpl` 27
Figure 28 Relationship between `Free` monad and `DT`. 28
Table 3 Functions implemented for the Free monad 28
Listing 19 Transliteration between universes of types 29
Listing 20 `corec` of progressive-approximation M-types given in Mathlib 29
Listing 21 Type-class `EquivP` for polynomial equivalents 30
Listing 22 Natural isomorphisms on MvFunctors 31
Table 4 Requirements and completion 32
Figure 29 Cumulative performance for stream destructing 33
Figure 30 Per-layer performance for stream destructing 34
Listing 23 LCNF for functions on the AltRepr Type 34
Listing 24 Futumorphic unfolding lemma 35
Listing 25 Interlace constant 36
Listing 26 Stutter constant 36
Listing 27 Run length decoding 36
Table 5 Comparison of coinductive libraries 37
Figure 31 Evenness inductive predicate, from first principles 40
Figure 32 Import graph of the lean component of the project as of 2026-04-04 45

Chapter 1: Introduction

Lean [1] is a pure and total functional language, meaning all functions must terminate. Problems such as program simulation or production of streams of data often require leaving the finite. We refer to data like this as *coinductive*. It does not make sense to require an infinite stream of data to terminate. Researchers working on program verification are also interested in this, they use structures called interaction trees as a denotational semantic [2].

Lean has two implementations of coinductive data types [3], [4], [5], [6]. These are implemented as a series of progressive approximations. As a limitation of this encoding, unfolding a layer of a coinductive type, takes time proportional to the depth of the layer. A consequence of this is that getting a stream to depth n takes $\mathcal{O}(n)$ time. This gets dramatically worse as you map streams, becoming intractable for most programs. An alternative encoding stores a generating function, parameterised by some carrier, and an initial state. This is what I will refer to as the state-machine encoding. By construction, destructuring will be $\mathcal{O}(1)$. The downside of this definition is that it is impredicative, and therefore becomes harder to use and reason about.

This dissertation is the final piece needed to complete state of coinductive data types in Lean. We have macros for defining [4] and writing functions [6] on coinductive data types. This dissertation is the first time anyone has decided to try and tackle the performance of library coinductive types. As a result of this, the path is now set to make a usable coinduction library.

I will implement the state-machine encoding of M-types, prove it is equivalent to the progressive approximation encoding, and use this to construct an efficient coinduction library. Additionally, we will implement interaction trees, with an equivalence relation that can be used to find contextually equivalent programs.

Chapter 2: Preparation

In this section I will discuss prerequisites for the implementation of the dissertation. I will discuss the background needed to understand the state-machine encoding, and other relevant topics.

2.1 Dependent type theory

Given functions between sets $A, B : \mathbf{Type}$, $A \to B$, we can generalize this by changing $B : \mathbf{Type}$ to be a function $B : A \to \mathbf{Type}$, then letting us write the dependent function $(x : A) \to B(x)$ (Π-type). We can also do the dual construction, and get dependent pairs (Σ-type). The inference rules for these can be seen in Figure 1. If you are unfamiliar with dependent types, it is often helpful to view them through this relationship. For a more in-depth look at dependent type theory, read the HoTT Book Section 1 up to Chapter 6 [7].

$$\frac{A : \mathbf{Type} \qquad B : A \to \mathbf{Type}}{(x : A) \to B(x)}\,\Pi\text{f} \qquad \frac{\Gamma, x : A \vdash v : B(x)}{\Gamma \vdash \lambda x.v : (x : A) \to B(x)}\,\Pi\text{i} \qquad \frac{\Gamma \vdash f : (x : A) \to B(x) \qquad \Gamma \vdash v : A}{\Gamma \vdash fv : B[x \;/\; v]}\,\Pi\text{e}$$

(a) Π-forming rule (b) Π-intro rule (c) Π-elimination rule

$$\frac{A : \mathbf{Type} \qquad B : A \to \mathbf{Type}}{(x : A) \times B(x)}\,\Sigma\text{f} \qquad \frac{\Gamma \vdash a : A \qquad \Gamma \vdash b : B[x \;/\; a]}{\Gamma \vdash \langle a, b, \rangle : (x : A) \times B(x)}\,\Sigma\text{i} \qquad \frac{\Gamma \vdash v : (a : A) \times B(a) \qquad \Gamma \vdash a : A, b : B[x \;/\; b] \vdash x : C}{\Gamma \vdash \text{let } \langle a, b \rangle := v \text{ in } x : C}\,\Sigma\text{e}$$

(d) Σ-forming rule (e) Σ-intro rule (f) Σ-elimination rule[6]

Figure 1: Inference rules for Π and Σ types

These rules are analogous to the rules for $\forall, \exists$ from the natural deduction system. In fact, they are related through the Curry-Howard correspondence as seen in Figure 2. Going from dependent types to the propositional rules, we must equate all proofs/terms. This process is called propositional truncation, as we truncate away data [7] (Chapter 3.7). The other direction 'categorification'. Categorification is the process of finding a categorical object that truncates down to the logical structure. S. Awodey [8] is a great resource on this.

$$\Pi, \Sigma \underset{\text{Categorification}}{\overset{\text{propositional truncation}}{\rightleftharpoons}} \forall, \exists$$

Figure 2: Curry-Howard correspondence

2.2 Universe levels

Consider Russell's Paradox `let R := { x | x ∉ x } in R ∈ R`. A way to resolve this is by making a hierarchy of sets `Set 0 : Set 1 : ...` .

Definition 1: A **universe** is a type, whose terms are themselves types [9].

We also have a paradox like this for types, Girard's paradox, which can be seen in [10] (Lecture 20). For this we make a hierarchy of types `Type 0 : Type 1 : ...`[7].

[6] Technically this is not the most general eliminator for Sigma's, as C can depend on the content of the type. For symmetry and simplicity I omit this.

[7] There are a few different semantics for doing this (universes à la Russell, Tarski, Coquand). It would be beyond this dissertation to go into this

Definition 2: We call an index into the hierarchy of universes a **universe level**. These universe levels form a join semilattice with a successor.

Definition 3: A definition is **universe polymorphic** if it uses some free universe levels $\mathcal{U}$.

Through this dissertation universe levels are written calligraphically ($\mathcal{U}$, $\mathcal{V}$, $\mathcal{W}$). Given types `X : Type` $\mathscr{u}$ and `Y : Type` $\mathscr{v}$, we have that `X × Y`, `X ⊕ Y` and `X → Y` all reside in `Type max` $\mathscr{u}$ $\mathscr{v}$. Σ- and Π-types also behave like this. Bumps are only introduced when objects hold a `Type` $\mathscr{u}$.

Example: Given two types `X : Type 2` and `Y : Type 3`, the function space between these two types resides at the level `max 2 3`, that is `X → Y : Type 3`.

If we have a type `X : Type` $\mathscr{u}$ and we want it to reside it some universe `max` $\mathscr{u}$ $\mathscr{v}$, we need to make a type-constructor `ULift.{`$\mathscr{v}$`} : Type` $\mathscr{u}$ `→ max` $\mathscr{u}$ $\mathscr{v}$. Working with `ULifts` can quickly become verbose. To avoid this many logics add a form of subtyping on universe hierarchies. Computationally this behaves like `ULifts`, but hides the lift.

Definition 4: A logic is **cumulative** if for some `X : Type` $\mathscr{u}$, then we also definitionally have `X : Type max` $\mathscr{u}$ $\mathscr{v}$ for any $\mathcal{V}$.

Many problems will be caused throughout this dissertation by universe levels. Therefore often I will simply give a less general, more universe stable type.

2.3 Lean

Lean [1] is a popular dependently typed proof assistant. Lean is known for many reasons, 3 of them are:
1. Lean has Mathlib [3], one of the world's largest repositories of formalized mathematics Section 2.3.1.
2. Lean has a rich tactic language in which most proofs are written Section 2.3.2.
3. Lean has a compiler, and can be used as a general purpose programming language Section 2.3.3.

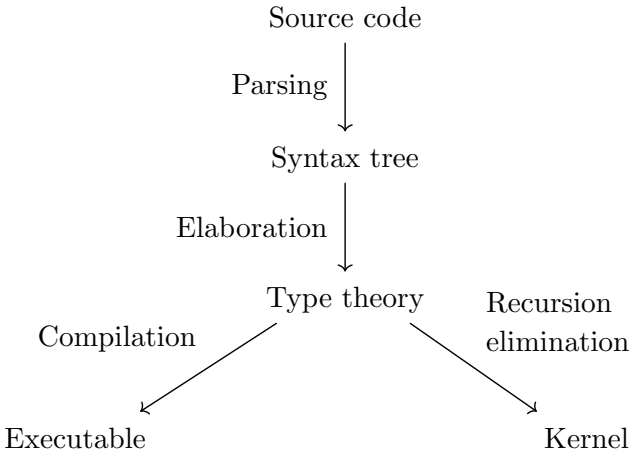


Figure 3: Lean compiler layout[8]

When talking about Lean we also use the term *definitionally equal* to mean β,η-equivalent, and *propositionally equal* if two terms are identified by a generalized algebraic data type. Additionally, we have *heterogeneous equality* where the types are only propositionally equal.

[8]Taken from https://lean-lang.org/doc/reference/latest/Elaboration-and-Compilation/

2.3.1 Mathlib

One of the primary use-cases of Lean is for the vast mathematics library known as Mathlib. Mathlib has many subcomponents in it such as category theory, analysis and geometry. It also has a full formalization of the QPF paper [5] done by J. Avigard and A. Keizer. This includes definitions of polynomial functors Section 2.4 which we will use as part of the implementation.

2.3.2 The tactic language

Many languages have metaprogramming features. Lean has a very complex compiler that is able to provide type information at certian metaprogram expansion. This is possible by taking place at during the elaboration stage as seen in Figure 3. Tactics are a form of metaprogramming aimed at constructing proof terms. Some of the tactics used in this project are:

1. `refine term` / `exact term` specify a term from term-mode rather than a tactic,
2. `intro var` becomes `refine fun var ⇒ ?_`,
3. `apply term` becomes `refine term ?_`,
4. `change sig` definitional type unfolding `refine (?_ : sig)`,
5. `rw [e₁, e₂]` rewrite, replacing occurances of the left hand side of equalities with the right hand side,
6. `simp [e₁, e₂]` simplifies expressions using passed equalities,
7. `induction obj` do an inductive case split,
8. `cases obj` do a case split,
9. `rcases obj with pattern` recursive case split,
10. `by_cases` law of excluded middle,
11. `ext` extensionality,
12. `congr` congruence,
13. `grind` SMT-solver,
14. `sorry` is a placeholder 'sorry I could not solve this'.

2.3.3 The Lean compiler

We have a dichotomy between the logical and computation sides of a proofs/programs as seen in Figure 3. A classic example of where this occurs is in compiling the elimination principle for strong-induction[9]. We talk about this being a difference between the kernel (the small safe piece of code we believe to be correct) and the compiler (which is much larger and more complex).

Lean requires all induction to be structural, so to do induction over a general well-founded relation, we need some inductive data type (Section 2.5), Lean calls this `Acc` and defines it as seen in Figure 4a. On this Lean defines a function `WellFounded.fixF` (Figure 4b), which allows for strong induction. This function is quadratic in its runtime, therefore as a consequence the kernel takes quadratic runtime to execute.

[9]This is now, for the case of natural numbers, fixed as of Lean 4.27. https://www.youtube.com/watch?v=LOUbbiV0mWc https://lean-lang.org/doc/reference/latest/releases/v4.27.0/

```
inductive Acc {α : Sort u} (r : α → α → Prop) : α → Prop where
  /--
  A value is accessible if for all `y` such that `r y x`, `y` is also accessible.
  Note that if there exists no `y` such that `r y x`, then `x` is accessible.
  Such an `x` is called a _base case_.
  -/
  | intro (x : α) (h : (y : α) → r y x → Acc r y) : Acc r x
```

(a) Definition for `Acc` taken from Lean's GitHub [1]

```
def WellFounded.fixF
    {α : Sort u} {r : α → α → Prop} {C : α → Sort v}
    (F : (x : α) → ((y : α) → r y x → C y) → C x) (x : α) (a : Acc r x)
    : C x := ...
```

(b) Type signature of `WellFounded.fixF`

Figure 4: Some strong-induction primitives from Lean

It would be unacceptable for a program using strong recursion to be quadratic for a general purpose programming language, therefore the compiler removes the usage of `fixF`, and generates the expected calls in the intermediate representation (LCNF).

We can see that given the source-code Figure 5a, it generates a definition Figure 5b with `Nat.fix` (a variant of `fixF`), but in the output LCNF we get Figure 5c, which does not contain any mention of `Nat.fix`, but we see the recursive call, demonstrating how the logical and computation definitions diverge.

```
set_option trace.Compiler.result true in
@[semireducible]
def gcd (a b : Nat) : Nat :=
  if h' : b = 0 then a
  else
    have : a % b < b := Nat.mod_lt a
      (Nat.pos_of_ne_zero h')
    gcd b (a % b)
termination_by b
```

(a) GCD source code

```
/--
info: def gcd._unary : (_ : Nat) ×' Nat →
Nat :=
WellFounded.Nat.fix (fun x ⇒
PSigma.casesOn x fun a b ⇒ b) fun _x a ⇒
  PSigma.casesOn (motive := fun _x ⇒
    ((y : (_ : Nat) ×' Nat) → InvImage
(fun x1 x2 ⇒ x1 < x2) (fun x ⇒
PSigma.casesOn x fun a b ⇒ b) y _x → Nat)
→
      Nat)
    _x
    (fun a b a_1 ⇒
      if h' : b = 0 then a
      else
        have this := …;
        a_1 ⟨b, a % b⟩ …)
    a
-/
#guard_msgs in
#print gcd._unary
```

(b) generated kernel code

```
[Compiler.result] size: 9
  def gcd a b : tobj :=
    let _x.1 := 0;
    let _x.2 := Nat.decEq b _x.1;
    cases _x.2 : tobj
    | Bool.false ⇒
      let _x.3 := Nat.mod a b;
      dec a;
      let _x.4 := gcd b _x.3;
      return _x.4
    | Bool.true ⇒
      dec b;
      return a
```

(c) Generated intermediate representation

Figure 5: Gcd as Source, Kernel and LCNF

This general trick inspired the project, as it demonstrates that there is precedent in Lean4 to have separate logical and executional models. This is further highlighted by the existence of the `@[implemented_by]` attribute, which lets the programmer off-load the runtime behaviour of a function to another possibly unsafe definition. When a definition is marked as `@[implemented_by x]`, the code-generator treats it as `x` when compiling the code. Safety of programs using `@[implemented_by]` is a complex endeavour to verify, as Lean has a tactic called `native_decide` that runs the generated code outside of the kernel.

2.4 Polynomial functors

Definition 5: A **functor** is pair of a map `(α : Type) → F α` sending types to types, and a map between types to functions between the functor at those types `(α β : Type) → (α → β) → F α → F β`, such that the map distributes over composition and preserves identities.

Example: An example of a functor in programming is `Lists`, given a `List String` we can use the function `len : String → Nat`, to make a function `map len : List String → List Nat`:

Polynomial functors are functors with a collection of constructors (the head type), where the children correspond to how many of the polymorphic argument are wanted. Alternatively they are functors generated from taking `Sums` and `Products` over `constants` and projections. The formal definition can be seen in Figure 6.

Definition 6: A **univariate polynomial functor** $P(\alpha)$ is given by: a head-type H, along with a child family c_h parameterised by the head $h : H$. A polynomial functor applied at type α, is a sigma-type of the head, and the child family [11]. As seen in Section 2.1, the object of this type will be a pair of a head and child functions.

$$P(\alpha) \triangleq (h : H) \times c_h \to \alpha$$

Figure 6: Formal definition of polynomial functor

This dissertation will mainly focus on multivariate polynomial functors. Instead of having a single child, multivariate polynomial functors have a n-tuple of children for an n-polynomial functor.

Polynomial functors have all finite products (Figure 8, Figure 9) and coproducts (Figure 8, Figure 10). Polynomial functors have fixed points corresponding to `inductive`s, and cofixed-points as `coinductive`s [4], [5]. In the figures there are ellipsis, in reality multivariate polynomials operate on Type-vectors which come from the category formed by the products $\mathbf{Type}^n$ for some $n : \mathbb{N}$, these are not ergonomic to work with.

2.4.1 Common polynomials

Some examples of common polynomials are projection (Figure 7), constant functors (Figure 8), binary products (Figure 9), and binary sums (Figure 10). One crucial and complex polynomial not shown here is composition. Composition takes an n-polynomial, and n lots of m-polynomials, and outputs an m -polynomial. For example, taking the 3-polynomial F and 3 2-polynomials G_1,G_2,G_3, the constructor of the composition takes a $F(G_1\alpha\beta)(G_2\alpha\beta)(G_3\alpha\beta)$. The polynomial notation for this can be found in [3], [5]. For those familiar, the composition polynomial works exactly like composition in primitive recursive functions.

```
inductive Prj (i : Fin n) (a_1 ... a_n : Type)
  where
  | mk (t : a[i])
```

(a) Inductive Notation

$$\lambda(i : \underline{n}).\langle 1, \lambda j \quad h.\ \text{if } i = j \text{ then } 1 \text{ else } 0\rangle$$

(b) Polynomial

Figure 7: Projection polynomial

```
inductive Const (T : Type) (a_1 ... a_n : Type)
  where
  | mk (t : T)
```

(a) Inductive Notation

$$\lambda(T : \mathbf{Type}).\langle T, \lambda i \quad h.0\rangle$$

(b) Polynomial

Figure 8: Constant polynomial

```
inductive Prod (A B : Type) where
  | mk : A → B → Sum A B
```

(a) Inductive Notation

$$\langle 1, \lambda i \quad ().1\rangle$$

(b) Polynomial

Figure 9: Product polynomial

```
inductive Sum (A B : Type) where
  | inl : A → Sum A B
  | inr : B → Sum A B
```

(a) Inductive Notation

$$\langle \mathbb{B}, \lambda \begin{pmatrix} 0 & \text{true} & \mapsto & 1 \\ 1 & \text{false} & \mapsto & 1 \\ _ & _ & \mapsto & 0 \end{pmatrix} \rangle$$

(b) Polynomial

Figure 10: Sum polynomial

2.5 Recursion and Inductive data type

Definition 7: An **inductive data type**, is a type with a collection of constructors, and an eliminator consuming the tree and producing some data. Objects of inductive data types are **well-founded** trees of data.

Recalling algebraic data types from languages such as OCaml, Haskell and Rust. These are structures freely generated from a set of constructors. The classic example is a list, given as two constructors, `val cons : 'a → 'a list → 'a list` and `val nil : 'a list`, and a way to consume lists `val fold : 'a → ('b → 'a → 'a) → 'b list → 'a`. We write this as seen in Listing 1. If we restrict these data types to well-founded trees, these are exactly inductive data types[10]. Any pure function that terminates must return a finite tree of constructors. Fold is guaranteed to terminate for any finite tree

In Lean, recursive functions are compiled into a call to some recursor for a type. For lists this recursor is a dependent generalization of `fold`. All structurally recursive functions can be compiled into a call to `fold`. We also have two operations `mk : F (M F) → M F` and `dest : M F → F (M F)`. These form an equivalence (J. Lambek [12] Corollary 2.4), giving us the result that the cofix-point of `F`, written as `M F`, is equivalent to `F (M F)`. In practice this means we can unfold `M F`.

```
type ('a, 'b) listF = Nil | Cons of 'a * 'b
type 'a list = Mk of (('a, 'a list) listF)

let mk x = Mk (x)
let dest = function | Mk v → v
let rec fold bh ih = function
    | Mk(Nil) → bh | Mk(Cons(h, t)) → ih h (fold bh ih t)
```

Listing 1: Definition of list in OCaml

2.6 Corecursion and coinductive types

Definition 8: A **coinductive data type** is a type with a collection of constructors, and a generating function. Objects of a coinductive data types are potentially infinite trees of data.

Where inductive data types have a `val fold : (('a, 'b) f → 'b) → (('a, 'g) f as 'g) → 'b` (rec) to consume them, coinductive types have an `val unfold : ('b → ('a, 'b) f) → 'b → (('a, 'g) f as 'g)` (corec) to produce them. Intuitively, view it as if a fix F yields a bounded structure, then cofix F yields an unbounded structure[11]. Another way to view coinductive types are as a state-machine, here the corecursor takes center stage. We can say a coinductive data type is a data type generated by some corecursor state-machine.

2.6.1 In OCaml

We can make colists (lazy lists) by thunking (putting under a unit function) recursive occurrences, this thunking allows infinite structures to exist in finite memory. Then all we need to do is add a definition of `unfold`, `mk` and `dest` (Listing 2). We also have constructors and destructors forming inverses as for recursive data types[12].

[10] In OCaml, Haskell and Rust, data types do not correspond directly to inductive types since we can have non-well-founded trees, an example is `let rec degen = Cons((), degen)`.

[11] That is fix $(1 + (A \times -))$ gives lists, then cofix $(1 + (A \times -))$ gives potentially infinite lists.

[12] The dual of Lambek's theorem [12]

```
type 'a colist = CMk of (unit → ('a, 'a colist) listF)

let mk x = CMk (fun _ → x)
let dest = function | CMk v → v ()
let rec unfold gen v = CMk (fun _ →
    match gen v  with
    | Nil → Nil
    | Cons(h, t) → Cons(h, unfold gen t))
```

Listing 2: Definition of colist in OCaml

The keen eyed will notice and call out that `unfold` is not structurally recursive, nor even decreasing at all. This is the key difference between inductive types and coinductive types; coinductive types can be 'infinitely big'. Despite not being terminating, the definition is *productive*. Termination is having a complete structure in finite time, productivity is being able to always compute the next layer in finite time.

Definition 9: **Productivity** is the property of a corecursive function to always be able to compute the next layer in finite time.

2.6.2 Streams

Streams are similar to colists. The difference is they have no 'nil' constructor. An inductive data type of this form is empty[13], though it is inhabited for coinductive data types. Streams and colists are also common in general purpose programming, Java has Stream, Rust and Python have Iterator. This backs up our use cases put forward in the introduction. We will give concrete implementations of them for each of the encodings as follows.

2.6.3 The M-type

The cofixed-point of a polynomial functor `F`, is called an M-type[14] a possibly infinitely deep tree, in which each layer is an unfolding of some polynomial `F`. This means the implementations must have both a corecursor `corec : {α : Type} → (f : α → F α) → α → M F`, and a destructor `dest : M F → F (M F)`. These are the functions mentioned in Section 2.6.1. Often we also want to ship a way to `mk` a new layer, but we can define this from the two operators above.

There are different approaches to encode M-types in Lean. Prior art is the progressive approximation encoding (Section 2.6.3.1) (as seen in Mathlib), and this dissertation introduces the state-machine encoding (Section 2.6.3.2).

2.6.3.1 Progressive approximation encoding

A simple way to encode M-types, are as functions emitting finite and growing trees that must at every level agree. This corresponds to the limit over the natural number category as the dual of J. Adámek [13] (see section "Free algebra algorithm"). Agreement is given by them being the same up to the previous depth as seen in Figure 11. Agreement can be thought of as the structure 'crystalising', since it exclusively 'grows' from the crystal edge.

[13]See the following Lean

```
inductive SS (α : Type) where | cons : α → SS α → SS α
def SS.toEmpty : SS α → Empty | .cons _ tl ⇒ toEmpty tl
```

[14]Often phrased as types who's semantics are terminal F-coalgebra.

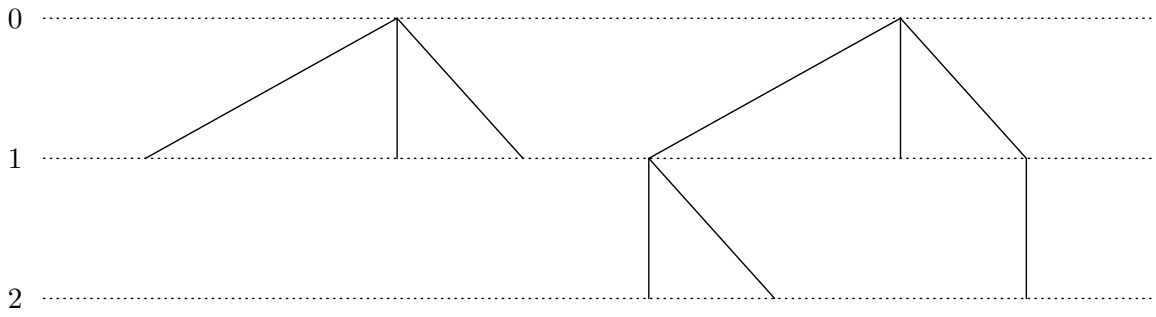


Figure 11: Agreement of two trees of height 1 and 2

In Lean, for a specific functor, we begin setting up the definition of approximation. With this we define the coinductive type as the agreeing functions.

```
def StreamF (α ρ : Type _) := α × ρ

inductive StreamF.Approx {α : Type} {base : Type} : {n : Nat} → {m : Nat}
    → n.repeat (StreamF α) base → m.repeat (StreamF α) base → Prop where
  | base  : @Approx _ _ 0 (m + 1) b v
  | consS : @Approx _ _ n m t₁ t₂ →
      @Approx _ _ (n + 1) (m + 1) ⟨h₁, t₁⟩ ⟨h₁, t₂⟩

structure PA (α : Type _) : Type 0 where
  obj : (n : Nat) → n.repeat (StreamF α) Unit
  cryst : ∀ n, StreamF.Approx (obj n) (obj (n + 1))
```

Listing 3: Definition of approx for stream

For the corecursor, we need to implement a function to emit the finite and growing trees, and a proof that the trees agree. This is also where the performance hit is incurred, as we have to regenerate the tree every time we want to reach the new depth.

```
def corec.o (gen : ρ → StreamF α ρ) (v : ρ) : (n : Nat) → n.repeat (StreamF α) Unit
  | 0 ⇒ .unit | n + 1 ⇒ (gen v).map id (o gen · n)
def corec (gen : ρ → StreamF α ρ) (init : ρ) : PA α where
  obj := corec.o gen init
  cryst n := by
    induction n generalizing init
    · exact .base
    case succ n ih ⇒
      unfold corec.o
      rcases gen init with ⟨a, b⟩
      change StreamF.Approx (n := n+1) (m := n+2) ⟨a, corec.o gen b n⟩ ⟨a, corec.o gen b (n + 1)⟩
      exact StreamF.Approx.consS (h₁ := a) (ih b)
```

Listing 4: Objects of the corecursor for streams

The destructor builds an instance of the functor, where the correct path is selected from the corecursor.

```
def dest (x : PA α) : StreamF α (PA α) := ⟨
  (x.obj 1).1,
  (x.obj ·.succ ▷.snd),
  fun n ⇒ by
    have := x.cryst (n+1)
    dsimp
    generalize (x.obj (n + 1)) = a, (x.obj (n + 1 + 1)) = b at this;
    rcases a; rcases b; rcases this with (_|ih)
    exact ih
⟩
```

Listing 5: Destructor for streams

Proving the coinduction principle (Section 2.7) is harder. We do not need to worry about implementing and proving this as Mathlib [3], [4] already provides an implementation of this.

2.6.3.2 State-Machine encoding

Given some polynomial functor `P`, the state-machine encoding of `M P` is given by: some *carrier* type `α : Type`, a function (coalgebra) `f : α → P α`, and some initial state `a : α`. Once this is done there are 2 key problems:

1. The object constructed is only weakly terminal and equivalently no coinduction principle,
2. the object resides in a higher universe.

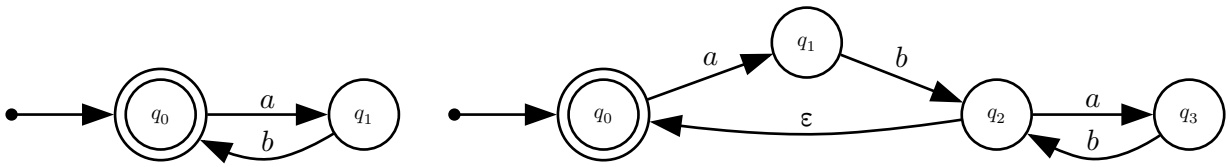
Focusing on the first 1st problem, this is solved by quotienting over bisimilarity. The reason this works has to do with how bisimilarity 'hides' the generating type. This makes destructuring the only operation you can do to access the type. The 2nd problem is tackled in Section 3.3.

2.7 Bisimilarity

Definition 10: **Bisimilarity** is an equivalence relation on coinductive types. Two coinductive objects are bisimilar if, for every possible choice in a structure A, there exists a corresponding choice in structure B, and vice versa, such that you can repeat this procedure.

Bisimilarity is not a well-founded relation. This means we need to define it as a coinductive predicate instead of an inductive one. Then the proof for this will be giving an invariant, showing the invariant is a bisimulation, and the initial values are contained within the invariant.

A classic example would be to check if the two state-machines are equal. In Figure 12 you see two state-machines for two different regular expressions. One could use bisimilarity to show these are the same. The invariant for this would probably be $(a = q_0 \wedge b = q_0) \vee (a = q_0 \wedge b = q_2) \vee (a = q_1 \wedge b = q_1) \vee (a = q_1 \wedge b = q_3)$. This also contains the initial state $a = q_0 \wedge b = q_0$. We also need to show the invariant is a bisimilarity which is a bit more involved.



(a) DFA for `(ab)*` (b) NFA for `(ab(ab*))*`

Figure 12: Two bisimilar state-machines

On streams, a bisimulation equates heads and relates tails. This can be seen in Listing 6. Coinductive types have coinduction principle `bisim : Bisim a b → a = b`. In Rocq this has to be an axiom, in Lean it is provable. M. P. Fiore [14] is an insightful read on this topic.

```
  Quotient.mk _ (.corec gen init)

@[simp]
theorem corec.hd {A Carr : Type _} {gen : Carr → A × Carr} {init : Carr}
    : (corec gen init).hd = (gen init).fst := rfl
```

Listing 6: Bisimilarity on streams

2.8 The Free Monad

As mentioned in Section 2.6, every corecursive function has to be productive. In the current library definition, this means we need to define the function using the corecursor `corec {β α} : (β → P (α ::: β)) → β → M P α`. This can be cumbersome as one can only define one layer of the coinductive data type at a time. One can image that after the first layer (which is required for productivity), we have a choice to either `recall` the corecursor from that point, or `continue` a deeper structure (visualized in Figure 13). A structure like this would allow for defining multiple layers of a coinductive.

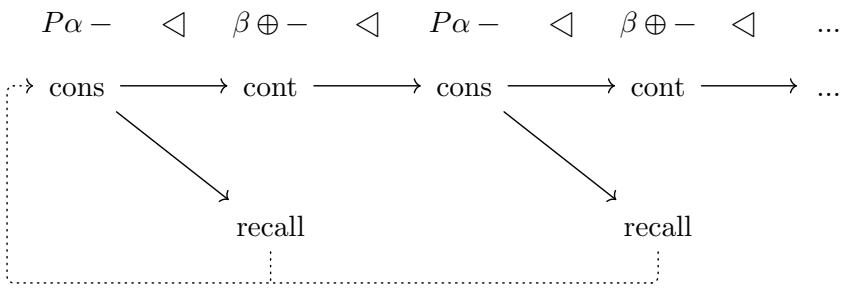


Figure 13: Choice points for a producing a stream

Making a structure like this is possible with a bit of setup.

Definition 11: A **Monad** `M` is a functor, with an additional operator `>>= : (α → M β) → M α → M β`, and a function `return : α → M α` satisfying the rules `(return ∘ f) >>= a = map f a`, `f >>= return v = f v` and `x >>= f >>= g = x >>= λx ↦ f x >>= g`.

Given a monad `M`, we can simply interpret it as a functor, but what if we want to go the other way; take a `F` and make the most general (or *universal*) monad out of it. The process for this involves keeping 'as much structure as possible', one way of doing this is have two constructor `return : α → Free F α`, and `bind : F (Free F α) → Free F α`. When doing this construction as a coinductive it is called the `Free`-monad. This has a bind operator that corresponds to pasting trees together. When referring to it, it will be generalized to a $n+1$-polynomial, and written `Free P α β`, the definition of this will be given in the implementation section.

Using the Free monad one can define a futumorphism `futu {β} : (β → P (α ::: Free P α β)) → M P α` [15], [16], [17]. This turns out to be exactly the structure wanted.

2.9 Interaction Trees

Denotational semantics stands as an alternative to operational semantics, where rather than having some relation of program steps, one maps syntax to some mathematical object which can be reasoned about directly. For lambda calculi we have categorical semantics, but for imperative languages it can be much harder to find a denotational object.

Interaction trees [2] are a form of coinductive free-monad, in which you have a set of visible effects. ITrees have three constructors, returning a value (monad unit), having a silent transition (for non-termination), and emitting a visible effect. A definition for this can be seen in Listing 7. The effects of the ITree correspond to the effects of the imperative language. Some examples could be printing, reading and writing to variables, and reading user input. ITrees have a 'monadic interpretation' where the effects are transformed to operations in some monad. Further, ITrees have an equivalence relation called weak bisimilarity, this is bisimilarity modulo silent steps, two objects are related if they have the same visible effects and return values.

We will speak more about ITrees in Section 3.5

```
coinductive ITree (E : Type → Type) (R : Type) : Type :=
| ret : R → ITree E R
| tau : ITree E R → ITree E R
| vis {A : Type} : E A → (A → ITree E R) → ITree E R
```

Listing 7: Notational defintion for ITrees

2.10 Methodology

Writing in a proof assistant is different to writing in a general purpose language, this is especially the case in Lean as it is a hybrid of both styles. A notable and immediate difference is the prevalence of

software testing of programs. Instead of having tests that can be flakey and sparse, in Lean you have checked proofs.

In a conventional programming language, you can follow Test Driven Development where you constantly refine the specification with additional tests, which you check as you go through the code. In Proof Driven Development (PDD) [18] you work similarly to TDD, but now when you have your function, instead of knowing it works for finite cases, you can be guaranteed that it satisfies the equation. Furthermore, the proofs can serve to enrich the main artifact. This will be seen in the ITree implementation as well as the Free monad. Lean makes working in this style a delightful experience using constructs such as `#check`, `#print` and `#eval`, one can see exactly what a function is and what it is doing. These can also be turned into conventional tests using `#guard_msgs in`, which raises a compile error upon an output changing. PDD was originally demonstrated for Idris, which in many ways is similar to Lean as it is a hybrid language, therefore I will apply this approach as I write the code.

Lean also has great automation features which I use at many points in the dissertation, these are the unification solver `exact?`, SMT solver `grind`, and the simplifer `simp`. These all stand as principled ways to develop programs and proofs that would otherwise be tedious for a human. I use these regularly to speed up the development process. When working with simp one must be aware of non-terminal `simp`s; `simp`s that do not close a goal, for these we use the `simp?` tactic to limit the set of available lemmas. The reason for this has to do with how adding more lemmas can break the normal form `simp` leads to. Lean's built in linter can handle this.

2.11 Starting point

I worked with meta-programming for polynomial during a UROP between Part Ia and Part Ib. This means I am aware of what the underlying structures are when it comes to the raw implementation. I also did a feasibility assessment of the project by seeing how the current polynomials respond to universe levels. This lead to me making 2 pull requests (#28095, #28279) to mathlib on `TypeVec` in preparation for my project. I also tried to merge #28112, an implementation of commutable Shrink to mathlib that later got reverted when it was found to be inconsistent. During the internship I also came up with the DeepThunk structure, but could not prove any theorems about it as I did not understand bisimilarity.

No code from the internship has been used in the dissertation. I tried to use the `Deepthunk` implementation, but this did not work and I had to rewrite it from scratch.

There are more minor refactoring pull requests towards the mathlib repository which don't change any behaviour but in general all of these can be *found on this link*. I Include these for completeness and transparency.

2.12 Tools used

To aid the development and communication process I used `git`, hosting the repository on GitHub. I also shipped a compiled version of this dissertation publicly, so my supervisors could read it and have it update live on the web. The dissertation is written in **typst** as it is a feature rich, well documented, and popular TEX alternative.

2.13 Requirement Analysis

> The key words “MUST”, “MUST NOT”, “REQUIRED”, “SHALL”, “SHALL NOT”, “SHOULD”, “SHOULD NOT”, “RECOMMENDED”, “MAY”, and “OPTIONAL” in this document are to be interpreted as described in RFC 2119.
>
> — S. O. Bradner [19]

Here the success criteria have become MUSTs, and improvements have become SHOULD and MAY. I have done this for the core and each of the extensions of the project.

2.13.1 State-Machine encoding (Core)

SM1 The SME Stream MUST be implemented in its high universe form.
SM2 The equivelence between SME Stream and PA Stream MUST be implemented.
SM3 The SME M MUST be implemented in its high universe form.
SM4 The SME M MUST be constant time under destructuring.
SM5 The equivelence between SME M and PA MUST be implemented.
SM6 The SME M MUST be able to express the NT monad, Section 2.13.2.

SS1 The SME M SHOULD have a coinduction principle.

SO1 The SME M MAY have a low universe shifted version Section 2.13.3.
SO2 The SME M MAY have an Interaction tree library Section 2.13.4.
SO3 The SME M MAY have a Productivity Transform Section 2.13.5.

SN1 The SME M MUST NOT have a syntax macro as in [4].
SN2 There MUST NOT be any work towards coinduction-up-to systems.

2.13.2 The delay monad (Core)

DM1 The delay monad MUST be implemented using the SME.
DM2 The delay monad MUST have a monadic bind and return.

DS1 The delay monad SHOULD be a lawful monad.

2.13.3 AltRepr Type (Extension)

AM1 The AltRepr Type MUST be implemented.
AM2 The AltRepr Type MUST have an eliminator.
AM3 The AltRepr Type MUST have the computational behaviour of the higher type.

AO1 The AltRepr Type MAY be zero-cost.

2.13.4 Interaction Trees (Extension)

IM1 ITrees MUST be implemented.
IM2 ITrees MUST have strong bisimilarity.
IM3 ITrees MUST have bind and return constructs.
IM4 ITrees MUST have KTrees (continuation trees).

IS1 ITrees SHOULD have flow combinators implemented.
IS2 ITrees SHOULD have a coinduction principle.
IS3 ITrees SHOULD be a lawful monad.

IO1 ITrees MAY have weak bisimilarity implemented.
IO2 ITrees MAY have the monadic interpretation.

IN1 ITrees MUST NOT implement the family of effects put forward in [2].

2.13.5 Free Monad (Extension)

FM1 Free Monad MUST have a futumorphism.
FM2 Free Monad MUST have an injector.

FS1 Free Monad SHOULD have proof-principles for unfolding the futumorphism.
FS2 Free Monad SHOULD have proof-principles for the injector.

FO1 Free Monad MAY have a universe polymorphic futumorphism.

FN1 Free Monad MUST NOT be heterogeneous [20].

Chapter 3: Implementation

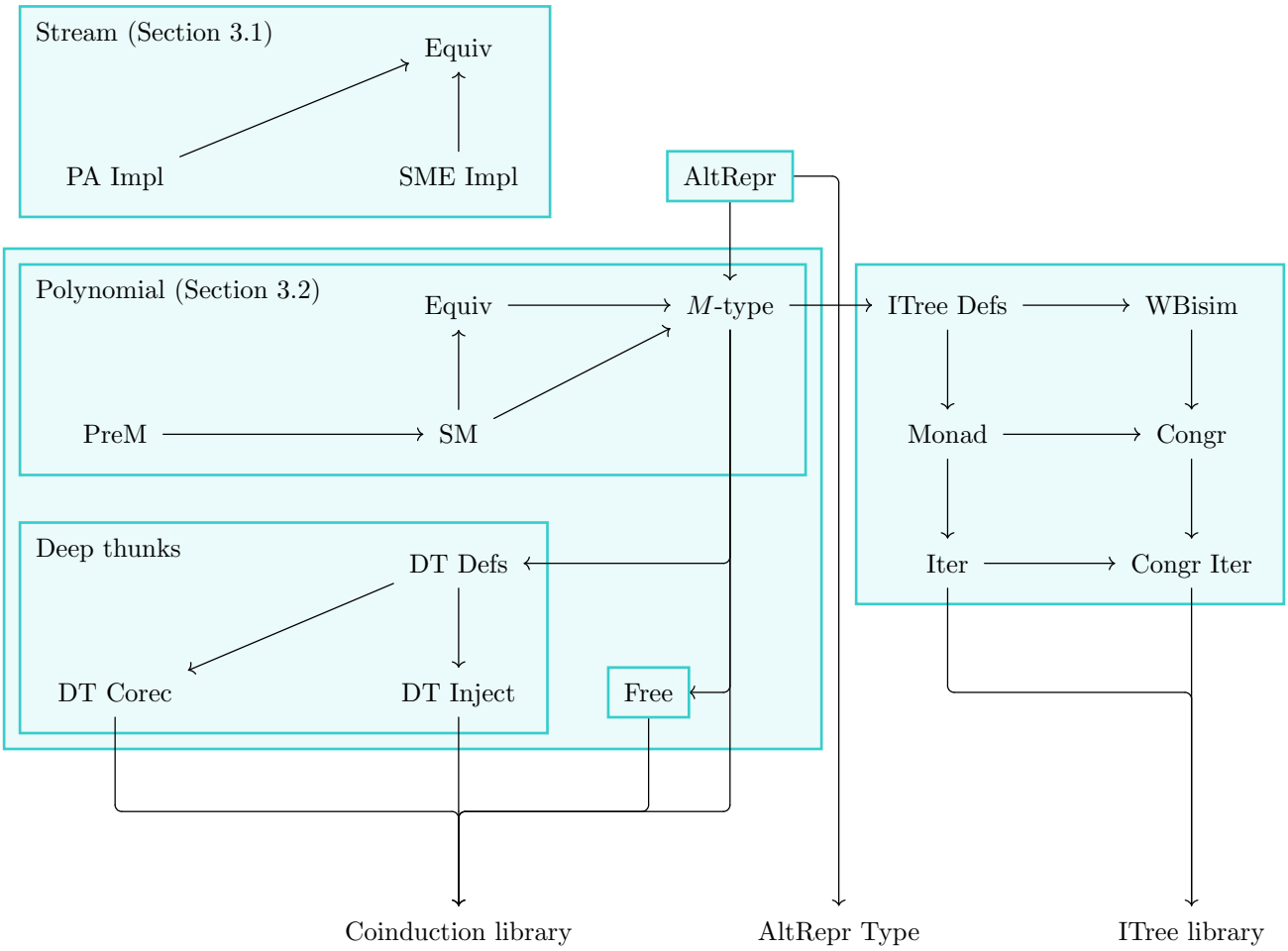


Figure 14: Birds-eye view of components of the project

The main component implemented in this dissertation is the state-machine encoding of the M-type `SM`. Further showing the equivalence between the state-machine encoded M-type `SM.{𝒰, max 𝒰 𝒱} P α`, and the progressive approximation encoding M-type `M.{𝒰} P α`. This will be used to implement a performant $\mathcal{U}$-universe State-Machine M-type, which has the efficiency of the `SM` encoding while being in the universe of the progressive approximation encoding.

3.1 Stream implementation

`sme/Sme/Stream/*` SM1, SM2

Proving the equivalence on M-types is hard, for this reason I will start by implementing it for the special case of streams. Streams are the text-book coinductive data type that most people know as mentioned in Section 2.6. Therefore I expected this to be pedagogical to implement.

3.1.1 State-machine streams

`sme/Sme/Stream/SDefs.lean` SM1

First setting up a class of prestreams, these correspond to the corecursive principle holding their type. Followed by the destructors `hd` and `tl` for streams. Care must be taken to ensure that the definition would work under a quotient as well.

```
structure PreStream (A : Type u) : Type (max u (v + 1)) where
  corec :: {Carr : Type 𝒱} (gen : Carr → A × Carr) (init : Carr)
def hd {A : Type 𝒰} (a : PreStream A) : A := (a.gen a.init).fst

def tl {A : Type 𝒰} (a : PreStream A) : PreStream A :=
  ⟨a.gen, (a.gen a.init).snd⟩
```

Listing 8: PreStream and destructors

Defining a bisimilarity relation; heads being equal and tails bisimilar as seen in Section 2.7. Followed by proving this was an equivalence relation (reflexive, symmetric, transitive). This would sets up a setoid on `PreStreams`.

```
coinductive Bisim
    (A : Type u)
    : PreStream A
    → PreStream A
    → Prop where
  | step {a b : PreStream A}
    (cont : Bisim a.tl b.tl)
    (h : a.hd = b.hd)
    : Bisim A a b
```

(a) Bisim definition

```
def Bisim.refl (x : PreStream A) : Bisim A x x :=
  ⟨Eq, (· ▸ .step rfl rfl), rfl⟩

def Bisim.symm (h : Bisim A a b) : Bisim A b a :=
  have ⟨r, his, rel⟩ := h
  ⟨(fun v ⇒ r v ·), (match his · with | .step ha hb ⇒ .step ha hb.symm), rel⟩

def Bisim.trans (hab : Bisim A a b) (hbc : Bisim A b c) : Bisim A a c :=
  have ⟨rab, hisab, hab⟩ := hab; have ⟨rbc, hisbc, hbc⟩ := hbc
  ⟨ (∃ b, rab · b ∧ rbc b ·),
    fun ⟨_, hax, hxb⟩ ⇒ match hisab hax, hisbc hxb with
      | .step hax e₁, .step hxb e₂ ⇒ .step ⟨_, hax, hxb⟩ (e₁.trans e₂),
    ⟨b, hab, hbc⟩ ⟩
```

(b) Equivalence relation

Figure 15: `PreStream` setoid

The definition of a state-machine `Stream` is then preobjects quotiented by this setoid. When lifting the destructors from `PreStreams` to `Streams`, I have to go through the lifting function of quotients. When lifting to a quotient one has to provide the lifting function, then a proof that the function is stable under the setoid. The corecursor is defined using the constructor for `PreStreams`, under the quotient constructor.

```
def hd : SStream A → A :=
  Quotient.lift PreStream.hd
    fun _ _ ⟨_, his, rab⟩ ⇒ match his rab with | .step _ h ⇒ h

def tl : SStream A → SStream A :=
  Quotient.lift (Quotient.mk _ ∘ PreStream.tl)
    fun _ _ ⟨r, his, rab⟩ ⇒ match his rab with
      | .step h _ ⇒ Quotient.sound ⟨r, his, h⟩
```

(a) Destructors for SStreams

```
def corec {A Carr : Type _}
    (gen : Carr → A × Carr)
    (init : Carr)
    : SStream A :=
  Quotient.mk _ (.corec gen init)
```

(b) `corec` for Streams

Figure 16: `corec` and destructors for SStreams

The next step is defining a coinduction principle; that bisimilarity implies equality. This is called `bisim` to align to convention with Mathlib. The proof of this proceeds by quotient soundness and can be found in `SDefs.lean`.

```
coinductive Bisim
    (A : Type _)
    : SStream A
    → SStream A
    → Prop
  | step
    {a b : SStream A}
    (cont : Bisim a.tl b.tl)
    (h : a.hd = b.hd)
    : Bisim A a b
```

(a) Bisim definition

```
theorem bisim {a b : SStream A} : Bisim A a b → a = b := by
  cases a, b using Quotient.ind₂; next a b ⇒
  rintro ⟨r, his, hhold⟩
  apply Quot.sound; change PreStream.Bisim A a b
  exact ⟨
    fun x y ⇒ r (.mk _ x) (.mk _ y),
    fun {x y} hxy ⇒ match his hxy with
      | .step htl hhd ⇒ .step htl hhd,
    hhold
  ⟩
```

(b) Coinduction principle

Figure 17: Coinduction principle for SStreams

3.1.2 Progressive approximation streams

`sme/Sme/Stream/PDefs.lean`

I implement the stream's base functor. Streams have one constructor (so a head of 1), and each of the families only hold one instance of the value (so child `λ i () ↦ 1`).

```
def PStream.Base : MvPFunctor 2 :=
  (PUnit, fun _ _ ⇒ PUnit)
```

(a) Functor

```
def PStream (A : Type u) :=
  MvPFunctor.M PStream.Base fun _ ⇒ A
```

(b) PStream

Figure 18: PStream definition

I define the destructors of streams by calling the child with the correct indices.

```
def hd (x : PStream A) : A :=
  x.dest.snd (.fs .fz) .unit
```

(a) Head of a PStream

```
def tl (x : PStream A) : PStream A :=
  x.dest.snd .fz .unit
```

(b) Tail of a PStream

Figure 19: PStream destructors

For symmetry, I define a syntactically identical bisimilarity relation on progressive approximation streams, and for this I prove a coinduction principle for PStreams of this relation. This proof proceeds by using the coinduction principle on general polynomial functors.

```
coinductive Bisim
    (A : Type _)
    : PStream A
    → PStream A
    → Prop
  | step {a b : PStream A}
    (cont : Bisim a.tl b.tl)
    (h : a.hd = b.hd)
    : Bisim A a b
```

(a) Bisimilarity of PStreams

```
theorem bisim {a b} : Bisim A a b → a = b := by
  refine fun ⟨r, his, holds⟩ ⇒ MvPFunctor.M.bisim _ r
    (fun a b rab ⇒ ⟨.unit, fun | .fz, .unit ⇒ hd a, fun .unit ⇒ tl a, ?_⟩) a b holds
  have ⟨htl, hhd⟩ := his rab
  refine ⟨fun .unit ⇒ tl b, ?_, ?_, fun .unit ⇒ htl⟩
  <;> refine Sigma.ext rfl ◁ heq_of_eq ◁ funext₂ fun | .fz, .unit | .fs .fz, .unit ⇒ ?_
  any_goals rfl
  change hd b = hd a
  exact hhd.symm
```

(b) Coinduction principle for PStreams

Figure 20: Coinduction principle for PStreams

The corecursor is lifted from the definition on M-types.

```
def corec {A Gen : Type _}
    (gen : Gen → A × Gen)
    : Gen → PStream A :=
  .corecU _ (fun g ⇒ ⟨.up .unit, fun
    | .fz,     .up .unit ⇒ .up (gen g).snd
    | .fs .fz, .up .unit ⇒ .up (gen g).fst⟩)
```

Listing 9: `corec` for PStreams

3.1.3 State-machine Progressive approximation equivalence

`sme/Sme/Stream/Equiv.lean` SM2

The functions of the equivalence between progressive approximation and state-machine Streams, are corecursors parameterised by the destructors of the other encoding. Proving that these are inverses is quite involved. I tried a few different relations trying to make it work. In the end, I landed on the straightforward equality as seen in Listing 10. This solves and made the entire proof small after cleaning.

Having done this, I now can handle the case of a generic polynomial functor. This turns out to be harder than I expected. The reader may notice the statement I proved is subtly different; rather than proving the statement for a universe polymorphic carrier `SStream.{𝒰, max 𝒰 𝒱} A`, here I prove it for a fixed universe `SStream.{𝒰, 𝒰} A`. This is not a problem here, but when it comes to instantiate the `AltRepr` (Section 3.3) type, the universe-polymorphic statement is required.

```
def Sme.Stream.equiv.{𝒰} {A} : SStream.{𝒰, 𝒰} A ≃ PStream A where
  toFun :=  PStream.corec SStream.dest; invFun := SStream.corec PStream.dest
  left_inv _x := SStream.bisim ⟨
    fun a b ⇒ a = .corec PStream.dest (.corec SStream.dest b),
    fun {a _b} (hab : a = _) ⇒ hab ▸ .step rfl rfl,
    rfl ⟩
  right_inv _x := PStream.bisim ⟨
    fun a b ⇒ a = .corec SStream.dest (.corec PStream.dest b),
    fun {a _b} (hab : a = _) ⇒ hab ▸ .step rfl rfl,
    rfl ⟩
```

Listing 10: Equivalence between progressive-approximation and state-machine streams

3.2 State-machine encoding of M-types

`sme/Sme/M/*` SM3, SM5

In this section, we will generalize from streams to the M-type. We will proceed in the exact same steps, First make a class of preobjects, then define a bisimilarity relation, finally making the state-machine M-type by quotienting bisimilarity.

As we are proving the universe-polymorphic equivalence, a universe lifting function is also needed.

3.2.1 Rephrasing bisimilarity for M-types

SS1

I implemented this dissertation using the definition of bisimilarity as given in Mathlib (Figure 21a). This definition turned out to be hard to work with as it introduced heterogeneous equalities, and required unfolding the definition. Later in my dissertation I discovered an equivalent formulation, using the universal property of the terminal object, since bisimilarity requires all but the last family to be equal, I equalize last families. Reducing the up-front cost of proving an M-type bisimilarity, as previously one had to instantiate a common head `a`, a shared child-family in all but the last argument `f` and two families in the last argument $\texttt{f}_1$ $\texttt{f}_2$. Finding these values was difficult and time-consuming as they usually required unfolding the definition. The definition given in Figure 21b ended up being much easier to use. One of the reasons for this is that all the proofs about mappings of polynomials can be applied in a `simp` call to get the preliminary proof. This makes bisimilarity statements easier to work with.

```
def MvPFunctor.M.bisim (R : P.M α → P.M α → Prop)
  (h :
    ∀ (x y : P.M α), R x y → ∃ a f f₁ f₂,
      M.dest P x = ⟨a, TypeVec.splitFun f f₁⟩ ∧
        M.dest P y = ⟨a, TypeVec.splitFun f f₂⟩ ∧
          ∀ (i : (P.B a).last), R (f₁ i) (f₂ i))
  (x y : P.M α) (r : R x y) : x = y := ...
```

(a) Mathlib definition

```
def bisim_map (R : P.M α → P.M α → Prop)
  (x : R a b)
  (h : ∀ s t, R s t →
    ∃ (x : (TypeVec.id ::: Function.const _ PUnit.unit) <$$> s.dest
      = (TypeVec.id ::: Function.const _ PUnit.unit) <$$> t.dest),
      ∀ v, R (s.dest.snd .fz v) ◁ t.dest.snd .fz
        ◁ cast (congr (congr rfl (Sigma.ext_iff.mp x).1) rfl) v)
  : a = b := ...
```

(b) New definition

Figure 21: Equivalent bisimilarity definitions

3.2.2 PreM

`sme/Sme/M/PreM.lean` SM3

To define PreM, I proceed like I did for Streams. We have to use the definition `corecU` over `corec`.

```
/-- Preobjects for the SME encoded M-Type -/
@[pp_with_univ]
structure PreM (P : MvPFunctor.{𝓤} (n + 1)) (α : TypeVec.{𝓤} n)
    : Type max u (v + 1) where
  corec ::
  {β : Type 𝓥}
  (gen : β → MvPFunctor.uLift.{𝓤, 𝓥} P (TypeVec.uLift.{𝓤, 𝓥} α ::: ULift.{𝓤, 𝓥} β))
  (g : β)
```

Listing 11: `PreM` definition

For the case of streams, I had two destructors `hd` and `tl`, in this case I only have `dest`. One would expect `dest` to have the signature `dest : PreM P α → P (α ::: PreM P α)`, but this is in-fact not possible, The reason this is `P : PFunctor.{𝓤} (n + 1)`, expects the applied arguments to all be in universe $\mathcal{U}$. As we can see in the definition above `PreM P α : Type max 𝓤 (𝓥 + 1)` for some $\mathcal{V}$, consequently `dest` has the signature `dest : PreM P α → ULift.{max 𝓤 (𝓥+1)} P (ULift α ::: PreM P α)`.

```
/-- Destructor of PreM types -/
@[inline, specialize P]
def dest (x : PreM.{𝒰, 𝒱} P α)
    : MvPFunctor.uLift P
      (TypeVec.uLift.{_, v + 1} α ::: PreM.{𝒰, 𝒱} P α) :=
  map (TypeVec.id ::: (PreM.corec x.gen ·.down))
    ◁ transliterate.{𝒰, v, v+1}
    ◁ x.gen x.g
```

(a) Destructor for `PreM`

```
@[simp]
theorem dest_corec
    (gen : β →
      MvPFunctor.uLift.{𝒰, 𝒱} P
        (TypeVec.uLift.{𝒰, 𝒱} α ::: ULift.{𝒰, 𝒱} β))
    (g : β)
    : (corec gen g).dest
    = (TypeVec.id ::: (corec gen ∘ ULift.down)) <$>
      (transliterate.{𝒰,v,v+1} ◁ gen g) := rfl
```

(b) Equation for destructoring a corec

Figure 22: Destructor and β-rule for `PreM`

In addition to `corec` and `dest`, I also need `ULifting` of `PreM`s. This is a function taking a `PreM.{𝒰, 𝒱} P α` to a `PreM.{𝒰, max 𝒱 𝒲} P α`. This will be used to state the full universe-polymorphic equivalence.

```
@[pp_with_univ, inline]
def uLift (a : PreM.{𝒰, 𝒱} P α) : PreM.{𝒰, max 𝒱 𝒲} P α :=
  .corec (fun x ⇒
    have v := a.gen x.down
    ⟨v.fst.transliterate, (.transliterate ::: .up ∘ .transliterate) ⊚ v.snd ⊚ .transliterate⟩
  ) (ULift.up.{w} a.g)
```

Listing 12: ULifting of `PreM`

I can now move on to define bisimilarity. I use the definition of bisimulation from Section 3.2.1.

```
def IsBisim (R : PreM.{𝒰, 𝒱} P α → PreM.{𝒰, 𝒱} P α → Prop) :=
    ∀ s t, R s t →
      ∃ h : (TypeVec.id ::: Function.const _ PUnit.unit) <$> s.dest
          = (TypeVec.id ::: Function.const _ PUnit.unit) <$> t.dest,
      ∀ v, R (s.dest.snd .fz v) (t.dest.snd .fz (cast (by
        rw [show s.dest.fst = t.dest.fst from (Sigma.ext_iff.mp h).1]
      ) v))
def Bisim : PreM.{𝒰, 𝒱} P α → PreM.{𝒰, 𝒱} P α → Prop :=
  (∃ R, IsBisim R ∧ R · ·)
```

Listing 13: Bisimilarity of `PreM` types

I then prove that it is an equivalence relation.

```
theorem refl (x : PreM.{𝒰, 𝒱} P α) : Bisim x x :=
  ⟨Eq, fun | _, _, .refl _ ⇒ ⟨rfl, fun _ ⇒ rfl⟩, rfl⟩
```

(a) Reflexivity

```
theorem trans (hab : Bisim a b) (hbc : Bisim b c) : Bisim a c :=
  have ⟨r₁, hisAb, rab⟩ := hab
  have ⟨r₂, hisBc, rbc⟩ := hbc
  ⟨(fun a c ⇒ ∃ b, r₁ a b ∧ r₂ b c),
    fun | x, z, ⟨y, rxy, ryz⟩ ⇒ (by
      have ⟨hxy, hxyrel⟩ := hisAb _ _ rxy
      have ⟨hyz, hyzrel⟩ := hisBc _ _ ryz
      refine ⟨hxy.trans hyz, fun v ⇒ ⟨_, (hxyrel v), ?_⟩⟩
      have := (hyzrel ◁ cast (by
        rw [show x.dest.fst = y.dest.fst
          from (Sigma.ext_iff.mp hxy).1]) v)
      rw [cast_cast] at this
      exact this
    ), ⟨b, rab, rbc⟩⟩
```

(b) Transitivity

```
theorem symm (h : Bisim a b) : Bisim b a :=
  have ⟨r, his, rab⟩ := h
  ⟨(fun a b ⇒ r b a), fun a b h ⇒
    have ⟨h, hrel⟩ := his _ _ h
    ⟨h.symm, fun v ⇒ by
      have := hrel ◁ cast (by
        rw [show b.dest.fst = a.dest.fst
          from (Sigma.ext_iff.mp h).1]) v
      rw [cast_cast, cast_eq] at this
      exact this
    ⟩, rab⟩
```

(c) Symmetry

Figure 23: Equivalence relation for `PreM` bisimilarity.

This gives `PreM` a setoid structure to instantiate the quotient.

3.2.3 State-machine M-type (SM)

`[UPSTREAMED],sme/Sme/{M/SM,HEq}.lean` SM3

I expected implementing the state-machine M-types to go as smoothly as implementing state-machine streams. Defining the corecursor was as simple. Universe lifting and other definitions were not.

The destructor of `SM`s will be given by quotient lifting as for streams. The function is also similar: take the preobject destructor and equalize deeper occurrences as to not leak the type. The stability of this function is proven by soundness of quotients for the corecursive case, and lifting the equality from bisimilarity for the other cases.

Proving the coinduction principle difficult. The reason for this, is heterogeneous equalities. Since Lean does not have injectivity of type constructors[15], often I need to solve them on a case-by-case basis. As a consequence I ended up writing a few crucial lemmas. These include `dcongr_heq` which I upstreamed to Mathlib. Therefore, this proof has both forward and backward sections. To save our sanity, I decided to rewrite it using Section 3.2.1. After doing this the proof falls out.

```
theorem bisim : Bisim a b → a = b := by
  rintro ⟨r, his, hhold⟩
  cases a, b using Quotient.ind₂; apply Quot.sound; change PreM.Bisim _ _
  refine ⟨fun x y ⇒ r (.mk _ x) (.mk _ y), fun s t rst ⇒ ?_ , hhold⟩
  clear *-his rst
  have ⟨hm, h⟩ := IsBisim.Alt_eq.mp his _ _ rst
  dsimp [dest] at hm h
  rw [MvFunctor.map_map, MvFunctor.map_map, TypeVec.appendFun_comp'] at hm
  exact ⟨hm, h⟩
```

Listing 14: Bisimilarity of `SM`-types

The next proof is stability of universe lifting of state-machine M-types, which involves lifting the carrier instead of any part of the polynomial functor. This is the universe lifting function I defined for PreM types. Universe lifting of this component is needed for proving the generalized equivalence I stated in Section 3.1.3.

3.2.4 State-machine Progressive approximation equivalence

`sme/Sme/M/Equiv.lean` SM5

Each of the directions will be given by each of their corecursors. Further universe changes must also be handled. Finding the functions that satisfied these universe changes was non-trivial, as `map` can not change universe levels, therefore I need new functions (`upMap`, `downMap` and `transliterateMap` Section 3.7.1). The resulting functions are given in Listing 15, here we can see the subtle universe changes both up and down.

```
let msm := MvFunctor.map (TypeVec.id ::: ULift.up.{𝓤, max 𝓤 𝓥 + 1}) ◦ SM.dest
let mpa := liftAppend.{𝓤, max 𝓤 𝓥} ◦ M.dest P ◦ ULift.down.{max 𝓤 𝓥, 𝓤}
let toFun  := .corecU P msm,
let invFun := .corec mpa ◦ ULift.up,
```

Listing 15: Functions of the equivalence

I now need to prove that the two functions are inverses. I was not familiar with bisimilarities when proving these statements, and tried many invariants, landing on forcing their equalities. With this, I prove it is a bisimilarity. I picked the head of one of the applications, and which are definitionally equal. This means the equality for the children is homogenous. The other direction followed analogously.

[15] An example is `(List A = List B) = (A = B)` is independent of Lean.

```
def equivP : SM.{𝒰, max 𝒰 𝒱} P α ≃ M.{𝒰} P α :=
  let msm := MvFunctor.map (TypeVec.id ::: ULift.up.{𝒰, max 𝒰 𝒱 + 1}) ∘ SM.dest
  let mpa := liftAppend.{𝒰, max 𝒰 𝒱} ∘ M.dest P ∘ ULift.down.{max 𝒰 𝒱, 𝒰}
  {
    toFun  := .corecU P msm,
    invFun := .corec mpa ∘ ULift.up,
    left_inv := fun x ⇒ SM.bisim (
      (· = (.corec mpa ∘ ULift.up ∘ .corecU _ msm) ·),
      by
        rintro _ ⟨⟨gen, g⟩⟩ rfl
        exact (MvPFunctor.liftR_iff _ _ _).mpr (
          .up (gen g).fst.down,
          fun
            | .fz, h ⇒ .corec mpa
              ◁ ULift.up
              ◁ M.corecU P msm
              ◁ corec gen (((gen g).snd Fin2.fz ∘ ULift.up) h.down).down
            | .fs s, .up h ⇒ (gen g).snd (.fs s) (.up h) ▷.down ▷ .up,
          fun
            | .fz, h  ⇒ .corec gen ((gen g).snd .fz (.up h.down)).down
            | .fs s, h ⇒ (gen g).snd (.fs s) (.up h.down) ▷.down ▷ .up,
          Sigma.ext rfl ◁ heq_of_eq ◁ funext fun | .fz | .fs _ ⇒ rfl,
          Sigma.ext rfl ◁ heq_of_eq ◁ funext fun | .fz | .fs _ ⇒ rfl,
          fun | .fz, h | .fs _, h ⇒ rfl,
        ),
        rfl
    ),
    right_inv := fun x ⇒ M.bisim _
      (· = (.corecU _ msm ∘ .corec mpa ∘ .up) ·)
      (fun a b ⇒ (· ▸ (
        (corec mpa ◁ .up b).dest.fst.down,
        ((M.dest P b).snd ·.fs),
        (M.corecU P msm ◁ (corec mpa ◁ .up b).dest.snd Fin2.fz ◁.up ·),
        (M.dest P b).snd .fz,
        Sigma.ext rfl ◁ heq_of_eq ◁ funext fun | .fz | .fs _ ⇒ rfl,
        Sigma.ext rfl ◁ .refl _,
        fun _ ⇒ rfl
      )))
      _ _
      rfl,
  }
```

Listing 16: Equivalence between state-machine and progressive approximation encoded M-types `equivP`

3.3 The Alternative Representation (AltRepr) Type

`sme/Sme/AltRepr.lean` AM1, AM2, AM3

Mathlib has a type called `Shrink`. This is a type which takes an equivalence between an `A : Type 𝒰` and a `B : Type 𝒱`, and returns a `Type 𝒰`. It has an equivalence `A ≃ Shrink A`. `Shrink` is defined by extracting a type using choice. Usage of `Classical.choose` can not be compiled.

```
class Small (α : Type 𝒱) : Prop where equiv_small : ∃ S : Type 𝒲, Nonempty (α ≃ S)

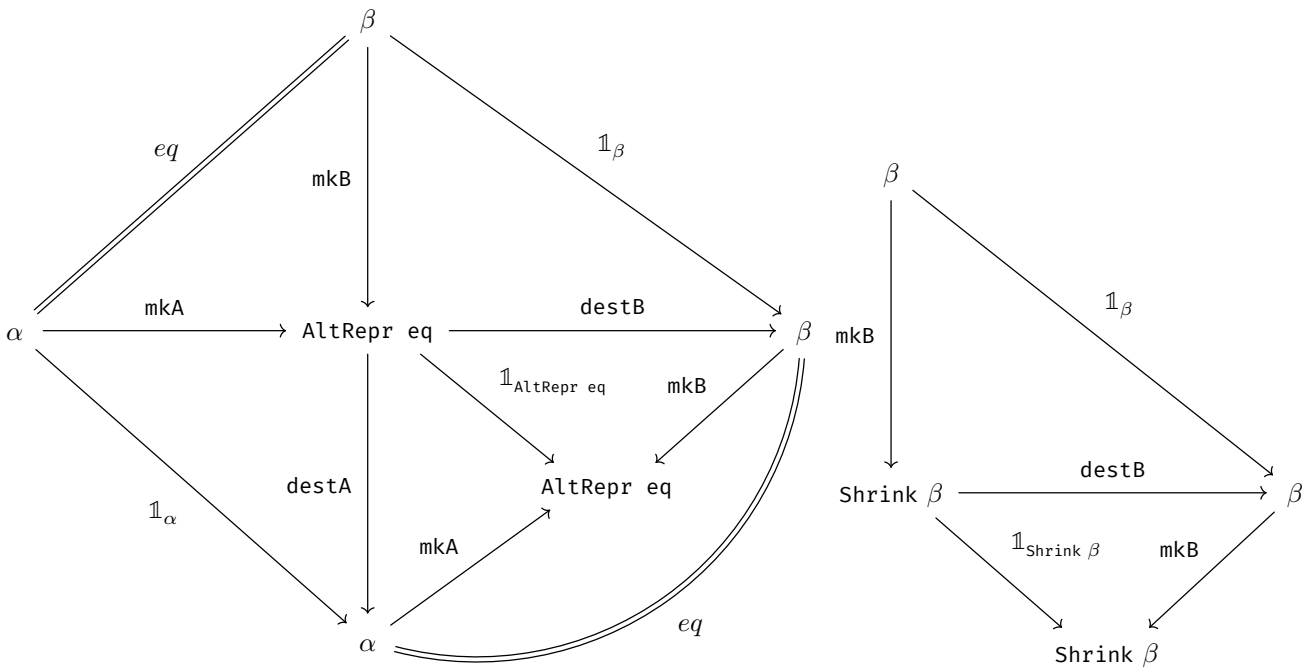
def Shrink (α : Type 𝒱) [Small.{𝒲} α] : Type 𝒲 := Classical.choose (@Small.equiv_small α _)

@[no_expose]
noncomputable def equivShrink (α : Type 𝒱) [Small.{𝒲} α] : α ≃ Shrink α :=
  Nonempty.some (Classical.choose_spec (@Small.equiv_small α _))
```

Listing 17: Mathlib definition of `Shrink`

I tried to make it computable by adding an `@[implemented_by]` to it, which turned out not to work as it made Lean unsound. Shrink satisfies univalence; given `Shrink A` and `Shrink B` along with `A ≃ B`, one can prove `Shrink A = Shrink B`. Aaron Liu from the Mathlib community used this in combination with quotients and trustCompiler to prove false. The proof by Aaron Liu can be found in Section 6.D.

From this, I decided to make my own variant I called the `AltRepr` type, as it gave a type an alternative representation. Given an isomorphism `eq : α ≃ β` for some types `α : Type 𝒰` and `β : Type 𝒰`, I define the type `AltRepr eq` to have two constructor-eliminator pairs `mkA`, `mkB`, `destA` and `destB`, satisfying the diagram seen in Figure 24a. This has an elimination principle satisfying expected β-rules with the two constructors. Through making the definition opaque there are still problems with universe levels. I solve this using a transliteration (Section 3.7.1). This let me eliminate into any universe independent of $\mathcal{U}$ and $\mathcal{V}$.



(a) Operations on AltRepr (b) Operations on Shrink

Figure 24: Universe transforming types.

3.3.1 The $\mathcal{U}$-universe State-Machine M-type

`sme/Sme/M/HpLuM.lean` SO1

After implementing the `AltRepr` type, it is time to put it to use. To do this, first I tried to use the equivalence, what I found was this leaked the universe of the carrier. I realised, I did not have to make a new equivalence for this, but paste the previous equivalence with itself. I call this "equiv cross universe" (`equivXU : SM.{𝒰, max 𝒰 𝒱} P α ≃ SM.{𝒰, max 𝒰 𝒲} P α`). One can observe this equivalence forms a transliteration (Section 3.7.1). Additionally one might wonder, does composing the equivalence with itself become a no-op? To which the answer is no, as we are parameterised in universe levels, so technically these are different functions. The performance of this equivalence is $\mathcal{O}(n^2)$. To avoid this, I decided to mark this as `@[implemented_by]` with a cast. Once this is done I could start implementing the $\mathcal{U}$-universe State-Machine M-type.

The $\mathcal{U}$-universe State-Machine M-type is the `AltRepr` type, parameterised by the equivalence given in the last section at universes `{𝒰 + 1, 𝒰}`. These universes prevent the carrier from being leaked. From here we define the corecursor of the same signature as in the `SM`, this uses a cocktail of universe changes, first one up using `SM.uLift`, then one down using `equivXU`, finally being passed into a `mkB` constructor of the `AltRepr` type. I also defined `dest : M P α → P (α ::: M P α)`. We observe this is a nicer signature than `SM.dest` and `PreM.dest`.

Next is proving the equations on the $\mathcal{U}$-universe State-Machine M-type. The first one was `dest_corec`, which has the expected signature. Followed by a coinductive principle. I also add an in-universe version of `corec` called `corec'`. This has the exact behaviour of the old non generalized corecursor on progressive-approximation M-types. In addition to this I add a corecursor unrolling lemmas and mapping lemmas. I then prove co-Lambek's theorem [12] and that the $\mathcal{U}$-universe State-Machine M-type is multifunctorial like the polynomial functor definition, along with destructor lemmas for map.

I add functionality to reason about polynomial equivalents (Section 3.7.3), this is the reason I picked the curried type-functions to be the `outParam`, as it lets these functions synthesize the correct curried type-functions without annotations.

I also found I needed a definition of ulifting of $\mathcal{U}$-universe State-Machine M-types. Like everything to do with universe levels, this turned out to be a challenge. I also prove naturality conditions (Section 3.7.4) of these.

Finally I prove one can transport a natural isomorphisms. This takes a natural isomorphism between `P` and `P'` and lifts it to a natural isomorphism between `M P` and `M P'`. This will be used when we start working with the futumorphism.

With this completed, I have a usable M-type library.

3.4 The delay monad

`sme/Sme/ITree/*` DM1

The delay monad [21] is an example of a coinductive data type. It has two constructors `tau : Delay A → Delay A` and `val : A → Delay A`. The values of this type will always be either some value of `tau`s then a `val`, or an a unique value `spin = tau spin`. One can think of these as `coroutines`, where interlacing destructing of taus, is interlacing computation.

I implement the delay monad by taking the cofixpoint of the Sum functor. Mainly I end up focusing on ITrees as they are a strict generalization of the Delay, This is adequate as `Delay A = ITree EmptyE A`, so it would be a waste to double up the implementation. I still implement the Delay as an exercise.

3.5 Interaction Trees

`sme/Sme/ITree/*` IM1, IM2, IM4, IS2

To define ITrees in Lean, we implement the functor, and prove results about polynomial equivalents on them. When KTrees are mentioned in the ITree paper [2], it is a function of the form `A → ITree E B`, for those familiar these are Kleisli morphisms [22].

```
CoInductive itree (E: Type -> Type) (R: Type): Type :=
| Ret (r: R)
| Tau (t: itree E R)
| Vis {A: Type} (e : E A) (k : A -> itree E R)
```

(a) Rocq definition from [2]

```
@[grind]
inductive Base (ρ R : Type _)
  | tau (r : ρ)
  | ret (t : R)
  | vis {A : Type u} (e : E A) (k : A → ρ)

inductive PHBase : Type max v (u + 1)
  | tau
  | ret
  | vis (A : Type u) (e : E A)

def PBase : MvPFunctor.{max v (u + 1)} 2 where
  A := PHBase E
  B | .ret ⇒ !![PUnit, PEmpty]
    | .tau ⇒ !![PEmpty, PUnit]
    | .vis A _ ⇒ !![PEmpty, ULift.{max v (u + 1), u} A]
```

(b) Parts of the Lean definition

Figure 25: Defintions of ITrees

ITrees have many operations and equations defined in the paper [2]. These can be seen in Table 1, here I annotate all functions I implement with superscript^{L}, from the paper [2] with^{R}, and the other implementation of ITrees for lean by Michael Sammler with ^{M}. A boon of the Lean implementations over Rocq, is that strong bisimilarity implies equality; the coinduction principle is provable in Lean. This fact is inherited from the M-type definition having a provable coinduction principle. This makes using some of the proofs easier.

Interaction Tree Operations

- `ITree E A : Type` [LRM]
- `tau : ITree E A → ITree E A` [LRM]
- `ret : A → ITree E A` [LRM]
- `vis {R} : E R → (R → ITree E A) → ITree E A` [LRM]
- `bind : ITree E A → (A → ITree E B) → ITree E B` [LRM]
- `trigger : E A → ITree E A` [LRM]

Heterogeneous weak bisimilarity

- `eutt (r : A → B → Prop) : ITree E A → ITree E B → Prop` [R]

Strong and weak bisimilarity

- `_ ≅ _ : ITree E A → ITree E B → Prop` [LRM]
- `_ ≈ _ : ITree E A → ITree E B → Prop` [LR]
- `∀ a b. a ≅ b → a = b` [LM]

Events and subevents

- `(e : E R)` [LRM] — R is the result of event e
- `E +' F` [LRM] — disjoint union of effects
- `class E -< F` [LRM] — E is a subevent of F
- `trigger [E ≺ F] : E ⇝ ITree F` [R]

Parametric functions

`E ⇝ F := ∀ X, E X → F X` [LR]

Monadic interpretation

```
interp [Monad M] [MonadIter M]
  : (E ⇝ M) → (ITreeE ⇝ M)
```
[LRM]

Table 1: Functions implemented in the dissertaton[L], in Rocq[R] [2] and M. Sammler[M] [6]

Structural Laws

- `tau t ≈ t` [LR]
- `t ≈ spin → t = spin` [L]
- `bind (tau t) k = tau (bind t k)` [LR*M]
- `bind (vis e k₁) k₂ = vis e λy ↦ bind (k₁ y) k₂` [LR*M]

Congruences

- `t₁ ≈ t₂ → tau t₁ ≈ tau t₂` [LR]
- `k₁ ≈ k₂ → vis e k₁ ≈ vis e k₂` [LR]
- `t₁ ≈ t₂ → k₁ ≈ k₂ → bind t₁ k₂ ≈ bind t₂ k₂` [LR]
- `k₁ ≈ k₂ → iter k₂ v ≈ iter k₂ v` [LR †]

Equivalence relation

- `a ≈ a` [LR]
- `a ≈ b → b ≈ a` [LR]
- `a ≈ b → b ≈ c → a ≈ c` [LR]

Monad Laws

- `(ret v) >>= k = k v` [LR*M]
- `t >>= ret = t` [LR*M]
- `x >>= f >>= g = x >>= (f · >>= g)` [LR*M]

Table 2: Equations implemented in the dissertaton[L], in Rocq[R] [2] and M. Sammler[M] [6][16]

[16] The annotations in the table are as follows:

[L] = Implemented in this dissertation

[R] = Implemented in [2]

[L] = Implemented in Michael Sammler

* = strong bisimilarity rather than equality in Rocq

† = proven using coinduction-up-to in Rocq

3.5.1 The ITree Monad

`sme/Sme/ITree/Monad.lean` IM3, IS1, IS3

ITrees form a monad, where the binding operation corresponds to pasting trees together, as seen in Figure 26; `bind` iterates through the tree and calls the function on seeing a `ret`. Figure 26 shows how biding $f(\circ) =$ `vis e₂ ◁ ...` over a tree `tau ◁ vis e₁ λ _ ↦ tau ◁ ret v` The `ret` constructor is the monads unit. To ensure it is lawful there is a set of proof obligations. These can be seen in Table 2.

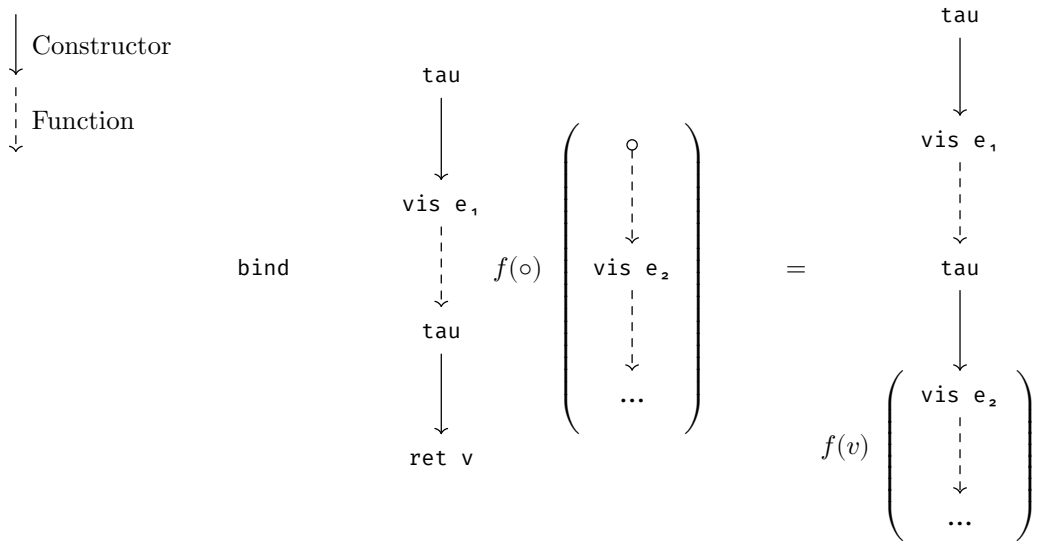


Figure 26: Visual representation of `bind` on ITrees

3.5.2 Weak bisimilarity

`sme/Sme/ITree/WBisim/*.lean` IO1

Bisimilarity is often not sufficient for proving program equivalences. This comes from the fact that sometimes we care about more than equality. For example, the program `println 10` and `println (10 + 0)` are *equivalent*, but not syntactically equal. We can note that the events produced by these programs are the same, yet one takes different steps to get there. This is where we consider weak bisimilarity instead. One can think of weak bisimilarity as an equivalence relation modulo silent steps; two objects with the same observable events but with a different tau count. The Rocq definition[17] and my definition differs. This is mainly because I decided to implement only homogeneous weak bisimilarity, and in Rocq they have coinduction-up-to. Coinduction-up-to is a technique where the prover does not have to provide the full bisimulation, but can rather use theorems to extend it [23], [24]. In my definition, I need to 'bake in' as many principles as possible. This is the reason for the `refl` constructor. In the definition I use `EStep` to mean some amount of taus, and `Step` to mean a productive step to some non-tau value. `EStep` forms a linear join-semilattice, `Step` inherits some of this structure, but is easier to work with. One of the key pieces of structure helping this be closer to coinduction-up-to is the ability to skip taus in a proof, this is done using the principle `skip : EStep a c → EStep b d → Invariant E A R c d → Invariant E A R a b`, which operates directly on the invariant for the bisimilarity. This solves problems similarly to how it is done in the paper, it gives a principle where we can simplify equations by shifting them along the taus.

[17] The syntax is slightly simplified for reading purposes here

```
CoInductive euttF : itree E A → itree E B → Prop :=
| EqRet a b (REL: r a b) : euttF (Ret a) (Ret b)
| EqVis {R} (e : E R) k1 k2 (REL: ∀v, euttF (k1 v) (k2 v)) : euttF
(Vis e k1) (Vis e k2)
| EqTau t1 t2 (REL: euttF t1 t2) : euttF (Tau t1) (Tau t2)
| EqTauL t1 ot2 (REL: euttF t1 ot2) : euttF (Tau t1) ot2
| EqTauR ot1 t2 (REL: euttF ot1 t2) : euttF ot1 (Tau t2).
```

(a) Rocq definition

```
coinductive WBisim'
    (E : Type u → Type u) (A : Type v)
    : Base E (ITree E A) A → Base E (ITree E A) A → Prop
    where
  -- This case only exists for refl spin spin,

  -- in fact it could be this,
  -- but that would be harder to prove things about
  | refl {a b c} : EStep a c → EStep b c → WBisim' E A a b
  | ret {a b v} : Step a (.ret v)
      → Step b (.ret v) → WBisim' E A a b
  | vis {V e a b a' b'}
```

(b) Lean definition

Figure 27: Weak-bisimilarity for ITrees

In the ITree paper [2], it is proven weak-bisimilarity is an equivalence relation. Proven identities can be found in Table 2. Out of these the ones marked with † have been proven using coinduction-up-to. These were particularly technical to prove and required defining helper predicates. The main instance of this is `iter`-congruence; if two KTrees are weakly bisimilar, the `iter` over them are equal. For this I had to define two predicates `IterTrace` and `IterCotrace` for productive and spinning `iter`s respectively. This proof was indirect and required the creative step of coming up with the (co)trace abstraction.

3.5.3 A formally verified optimization

`sme/Sme/ITree/WBisim/*.lean` IO1

One of the main use cases of Interaction Trees is program verification. I implement a simple imperative language IMP as seen in Listing 18a. I give it the denotational semantic as Listing 18b, then I define an optimization `simpl` Listing 18c on this. All `simpl` does is simplify constant expressions, the proof of this respecting the semantics is provided in the artifact, and is an example of how weak-bisimilarity can help prove an optimization correct.

```
inductive Imp (N : Type) : Type
  -- arith
  | add (a b : Imp N)
  | mul (a b : Imp N)
  | constN (n : Int)
  -- variables
  | var    (v : N)
  | assign (k : N) (a : Imp N)
  -- IO
  | print (a : Imp N)
  | seq (a b : Imp N)
  -- control
  | ite (c t f : Imp N)
  | while (c b : Imp N)
deriving DecidableEq, Repr
```

(a) `Imp` syntax

```
def sem {N} : Imp N → ITree (IOE ⊕, MapE N Int)
Int
  -- arith
  | .add a b ⇒ do
    let a ← sem a
    let b ← sem b
    return a + b
  | .mul a b ⇒ do
    let a ← sem a
    let b ← sem b
    return a * b
  | .constN n ⇒ .ret n
  -- var
  | .var v ⇒ .trigger ◁ .inr ◁ .getD v 0
  | .assign k v ⇒ do
    let v ← sem v
    let _ ← .trigger ◁ .inr ◁ .insert k v
    return 0
  -- effects
  | .print v ⇒ do
    let v ← sem v
    let _ ← .trigger ◁ .inl ◁ .print v
    return 0
  | .seq a b ⇒ do
    let _ ← sem a
    sem b
  -- control
  | .while c b ⇒ .iter (fun _ ⇒ do
      let c ← sem c
      if c ≠ 0 then
        let _ ← sem b
        return .inl .unit
      else
        return .inr 0
    ) Unit.unit
  | .ite c t f ⇒ do
    let c ← sem c
    if c ≠ 0 then sem t
    else          sem f
```

(b) `Imp` semantics

```
def simpl {N} (inp : Imp N) : Imp N :=
  flat ◁ pass inp
where
  flat : Int ⊕ _ → _
    | .inl v ⇒ .constN v
    | .inr v ⇒ v
  pass : _ → Int ⊕ _
    | .add a b ⇒
      match pass a, pass b with
      | .inl a, .inl b ⇒ .inl (a + b)
      | .inr a, b ⇒ .inr ◁ .add a (flat b)
      | .inl a, .inr b ⇒
        .inr ◁ .add (.constN a) b
    | .mul a b ⇒
      match pass a, pass b with
      | .inl a, .inl b ⇒ .inl (a * b)
      | .inr a, b ⇒ .inr ◁ .mul a (flat b)
      | .inl a, .inr b ⇒
        .inr ◁ .mul (.constN a) b
    | .constN n ⇒ .inl n
    | .print v ⇒
      .inr ◁ .print ◁ flat (pass v)
    | .var n ⇒ .inr ◁ .var n
    | .assign n v ⇒
      .inr ◁ .assign n ◁ flat (pass v)
    | .seq a b ⇒
      match pass a, pass b with
      | .inl _, b ⇒ b
      | .inr a, .inl b ⇒
        .inr ◁ .seq a (.constN b)
      | .inr a, .inr b ⇒ .inr ◁ .seq a b
    | .ite c t f ⇒
      match pass c, pass t, pass f with
      | .inl 0, _, f ⇒ f
      | .inl (.ofNat (_ + 1)), t, _
      | .inl (.negSucc _), t, _ ⇒ t
      | .inr c, t, f ⇒
        .inr ◁ .ite c (flat t) (flat f)
    | .while c b ⇒
      match pass c, pass b with
      | .inl 0, _ ⇒ .inl 0
      | .inl (.ofNat (_ + 1)), b
      | .inl (.negSucc _), b ⇒
        .inr ◁ .while (.constN 1) ◁ flat b
      | .inr c, b ⇒
        .inr ◁ .while c ◁ flat b
```

(c) `simpl`

Listing 18: `Imp` and `simpl`

3.6 Free Monad

Originally when considering the choice points for creating a variable layer polynomial functor (Section 2.8), I constructed what seemed to be the most obvious type. This structure became known as DeepThunks `DT P (α ::: β)`, and was given as the cofix-point of `M ((P α -) ◦ (Sum β -)))` (given in `sme/Sme/M/DT/Defs.lean`). I define `dtcorec {β} : (β → DT P (α ::: β)) → β → M P α`, an injector `inject β : M P α → DT P (α ::: β)`, and a flattener `flatten : DT P (α ::: M P α) → M P α`. These are extremely challenging to reason about. After much work I prove an unfolding lemma of the corecursor, where it corresponds to doing a mapping operation followed by flattening.

I found a similar function called a futumorphism. This uses the free monad to make a morphism `futu {β} : (β → P (α ::: Free P α β)) → M P α`. The relationship between the `Free` monad and `DT` can be seen in Figure 28. This demonstrates that `P (α ::: Free α β)` is exactly `DT`. A crucial difference is that `Free` has a well-defined notion of constructors and eliminator, while `DT` has nothing equivalent. This leads to a massive reduction in code size when switching to the `Free` monad. The implemented functions can be seen in Table 3.

$$\overbrace{\underbrace{(P\alpha\ -) \lhd (\beta \oplus -) \lhd (P\alpha\ -) \lhd (\beta \oplus -) \lhd \ldots}_{\text{DeepThunk}}}^{\text{Free monad}}$$

Figure 28: Relationship between `Free` monad and `DT`[18].

The definition given for deep-thunks used the carrier as one of the mapped parameters to the polynomial functor. I did not need to implement a dedicated map function, this was a mistake. The problem here is that any multivariate map can change the carrier. For all equations involving the multivariate map, had to be restricted to functions concatenated together. This also meant that working with binds became a mess for universe levels. Fundamentally we want to treat this as a different parameter so we can also use do notation. Proving facts about binds also became extremely tedious. We also did not have a monad unit as the structure of the base polynomial functor P can be arbitrary. Another problem was DeepThunks across universes required applying a sequence of natural transformations. These natural transformations had few equations other than naturality proven about them.

When implementing the Free monad, I apply each of these learnings. The definition used has the universe lifting built in, meaning we only have to reason about switching universes once (`dest_corec`). The carrier was also taken as a function parameter used to instantiate a constant. This solved the problem of maps being able to change the carrier, and at the same time allowed the free monad to satisfy the signature needed for the `Monad : (Type → Type) → Type` typeclass. This was also made even simpler by implementing the corecursor and the destructor using the standard Lean `Sums`. With these design decisions, I did not only implement the constructors (`recall` and `cont`) for the `Free` monad, but also a third constructor `cont'` which was in-universe allowing for some definitions to be simplified. For these I add a `cases` and `cases'` destructor. I also set up the corecursor to operate on `Sums` as well which made writing functions easy. The coinductive principle for M-types could also be specalized to the Free monad using relational lifting on `Sum` types. Building on the preexisting simp set for `Sums`, and marking the β-rules as `@[simp]`, many theorems became as clean as just invoking `simp`. The best examples of these can be seen in the mapping equations on constructors. These theorems are much cleaner than the corresponding theorems for `DeepThunks`.

Free Monad Operations

- `Free P α β : Type`
- `recall : β → Free P α β`
- `cont' : P (α ::: Free P α β) → Free P α β*`
- `dest' : Free P α β → β ⊕ P (α ::: Free P α β)*`
- `corec' : (γ → β ⊕ P (α ::: γ)) → γ → Free P α β*`
- `map : Free P α β → (β → γ) → Free P α γ`
- `bind : Free P α β → (β → Free P α γ) → Free P α γ`
- `inject β : M P α → Free P α β`
- `flatten : Free P α (M P α) → M P α`
- `futu' : (β → P (α ::: Free P α β)) → β → M P α*`

Monad Laws

- `(recall v) >>= k = k v`
- `t >>= recall = t`
- `x >>= f >>= g = x >>= (f · >>= g)`

Functor Laws

- `id <$> x = x`
- `(f ∘ g) <$> x = f <$> g <$> g`

Table 3: Functions implemented for the Free monad[19]

[18]Here $\lhd$ is application as seen in OCaml.

[19]The annotations in the table are as follows:

* = Cross universe version also implemented.

3.7 Machinery

Many large components of this project do not fit into any of the main components. This section is dedicated to this exact kind of component.

3.7.1 Transliteration

Definition 12: A **transliteration** over some parameter span[20] X, either some **Type** or more commonly over some universe $\mathcal{U}$, is a family of functions $t_{a,b} : X_a \to X_b$. Such that
1. the functions are closed under composition $t_{b,c} \circ t_{a,b} = t_{a,c}$[21],
2. the self-path is identificaion $t_{a,a} = \mathrm{id}$.

A family of functions, useful functions throughout this dissertation are what I call *transliterations*.

The trivial instantiation of a transliteration is a `cast`. Here we pick X as equal types. Obviously `cast aa = id` and `cast bc ∘ cast ab = cast (ab.trans bc)`. One could even say that a transliteration is a function that behaves like `cast`.

An example we will keep defining is universe transliteration, here we take $X = \mathrm{ULift}_{(-)}$, using this we define the transliteration as seen in Listing 19. This is the closest we get to cumilativity in Lean; through an application of the transliteration `transliterate : ULift.{v} α → ULift.{w} α`, we can use a type as if it is in any higher universe. We will define transliteration between universe-lifted polynomial functors, we will see more of this in Section 3.7.2.1.

```
universe u v w
variable {A : Type u} {x : ULift A} {a : A}

@[inline] def transliterate : ULift.{v} A → ULift.{w} A := ULift.up ∘ ULift.down

@[simp] theorem transliterate_idempotent
    : transliterate ∘ transliterate = transliterate (A := A) := rfl
@[simp] theorem transliterate_noop
    : transliterate x = x                                  := rfl
```

Listing 19: Transliteration between universes of types

Another major usage of a transliteration was used when defining the eliminator for `AltRepr`, I will discuss this more when we get to Section 3.3. This is where I first discover transliterations, and makes it possible to define a universe-polymorphic eliminator.

3.7.2 Expanding the progressive approximation theory

`[UPSTREAMED],sme/Sme/ForMathlib/{PFunctor/*,TypeVec.lean}`

During the feasability assesment, I noticed that in the current formalised theory of polynomial functors, the equivalence would not even type-check. This follows from the fact that the corecursor (Listing 20) requires that both α and β reside in $\mathcal{U}$. This would stop the corecursor having a state that is in a higher universe than the output. Solving this is done through the next two sections.

```
def corec : {α : TypeVec.{𝓤} n} → {β : Type 𝓤} → (g : β → P (α ::: β)) → β → M P α
```

Listing 20: `corec`[22] of progressive-approximation M-types given in Mathlib

3.7.2.1 Universe lifting of polynomial functors

The main problem caused here comes from the fact that Lean is not cummulative. This means it is impossible to express a universe heterogeneous typevector. In other words `α ::: β` is only typable

[20] This is not necessarily a type, as Lean does not have omega-types (Setω from Agda) [25].

[21] This is similar to saying it is an idempotent, but technically $t_{b,c}$ and $t_{a,b}$ are not the same function.

[22] https://github.com/leanprover-community/mathlib4/blob/7a60b315c7441b56020c4948c4be7b54c222247b/Mathlib/Data/PFunctor/Multivariate/M.lean#L152-L154

if `α : TypeVec.{𝒰} n` and `β : Type 𝒰`. The natural way of solving this is using the supremum in universe levels you get from `ULift : Type 𝒰 → Type max 𝒰 𝒱`. This means we can have `β : Type 𝒰` and `α : Type 𝒱`, then ulift both of them to a common universe `ULift α ::: ULift β : TypeVec.{max 𝒰 𝒱} (n+1)`[23].

The next hurdle we encounter is that polynomial functors are restricted to a universe level. Recall the definition from Section 2.4. Observe how for a `MvPfunctor.{𝒰} n`, we require that both the head and child reside in $\mathcal{U}$. This will also cause problems, as looking back at the definition of the corecursor, we will require P to be able to accept `ULift α ::: ULift β`. If we do not add the ability to lift P, the unifier will force $\mathcal{U} = \mathcal{V}$, thereby invalidating all the work we did in the previous section. We define it as `ULift P := ⟨ULift P.1, λ x ↦ ULift (P.2 x.down)⟩`. This definition also satisfies naturality. This works and now we can move on to generalizing the corecursor.

3.7.2.2 Generalizing the corecursor

`[UPSTREAMED],sme/Sme/PFunctor/ForMathlib/M.lean`

Now with all the work in the previous section, first we generalize a helper function[24], then we can define `corecU : {α : TypeVec.{𝒰} n} → {β : Type 𝒱} → (g : β → ULift P (ULift α ::: ULift β)) → β → M.{𝒰} P α`. Notably we are able to fit the object into $\mathcal{U}$ (this will not be the case for the state-machine encoding).

The functions `corecU` and `dest` satisfy an unfolding equation. This is more complex than it used to be as we now need to lower the universe before we continue with the mapping.

3.7.3 Polynomial equivalents

`sme/Sme/PFunctor/EquivP.lean`

Often when people talk about polynomial functors, they are referring to structures that are equivalent As in a `List` is not a pair of a head and child types, but it is equivalent to a polynomial of this form. For this I made a type-class `EquivP` (Listing 21) heavily inspired by how it was done in A. Keizer [4] and Mathlib. I took the definition of curried type-functions from QPFTypes [4] and implemented coherences for these for the common polynomials. I used this to make it cleaner to work with ITrees when I reached that point. A key difference is that I implemented it using the output curried type-functions as an outParam, this means that I can synthesize the curried type-functions from a given polynomial allowing for an easier user interface. Type class synthesis works similar to a prolog engine and `outParam` corresponds to the prolog mode `?`, whereas the default mode is `+`.

```
class EquivP (n : Nat)
    (F : outParam (CurriedTypeFun.{u, v} n))
    (P : MvPFunctor n)
    : Type max (u + 1) v where
  equiv : ∀ {t}, P t ≃ F.app t
```

Listing 21: Type-class `EquivP` for polynomial equivalents

3.7.4 Natural isomorphisms of MvFunctors

`sme/Sme/PFunctor/NatIso.lean`

When working with isomorphic Mv(P)Functors, it is often useful to require them to be natural. Formally this means an isomorphism `iso` satisfies `f <$$> iso x = iso (f <$$> x)`. To solve this I defined natural isomorphisms of multivariate functors Listing 22. The naturality of the inverse of the isomorphism is provable.

[23] Note we overload ULift as a notation to refer to lifting other structures as well.

[24] Done in PR ♯30400 to Mathlib

```
structure NatIso {n} (P Q : outParam (TypeVec.{v} n → Type u))
    [MvFunctor P] [MvFunctor Q] where
  equiv : ∀ {α}, P α ≃ Q α
  nat' {α β} {f : α ⟹ β} (x : P α) : f <$> equiv x = equiv (f <$> x)
```

Listing 22: Natural isomorphisms on MvFunctors

Chapter 4: Evaluation

This chapter discusses and evaluates each of the components of the dissertation. As mentioned in Section 2.10, I have been writing proofs verifying the correctness of my different components. I refer to these throughout this chapter. In addition to the implemented proofs, I will benchmark the state machine encoding, and read LCNF for the `AltRepr` type. There will be a comparison of coinductive and ITree implementations, and finally an assessment of the ergonomics of using the futumorphism.

An overview of all the requirements laid out can be viewed in Table 4.

Req.	Met	Evidence	Path
SM1	Y	Section 3.1	`Stream/SDefs.lean`
SM2	Y	Section 3.1.3	`Stream/Equiv.lean`
SM3	Y	Section 3.2.3	`M/Defs.lean`
SM4	Y	Section 4.1	`M/HpLuM.lean`
SM5	Y	Section 3.2.4	`M/Equiv.lean`
SM6	Y	Section 3.4	`NTMonad/Defs.lean`
SS1	Y	Section 3.3.1	`M/{HpLuM,SM}.lean`
SO1	Y	Section 3.3.1	`M/HpLuM.lean`
SO2	Y	Section 3.5	`ITree/Defs.lean`
SO3	Y	Section 3.6	`M/DT/*,M/Futu.lean`

(b) State-machine encoding

Req.	Met	Evidence	Path
IM1	Y	Section 3.5	`ITree/Defs.lean`
IM2	Y	Section 3.5	`ITree/Defs.lean`
IM3	Y	Section 3.5.1	`ITree/Defs.lean`
IM4	Y	Section 3.5	`ITree/Defs.lean`
IS1	Y	Section 3.5	`ITree/Defs.lean`
IS2	Y	Section 3.5	`ITree/Defs.lean`
IS3	Y	Section 3.5	`ITree/Defs.lean`
IO1	Y	Section 3.5.2	`ITree/Defs.lean`
IO2	Y	Section 3.5	`ITree/Defs.lean`

(a) ITrees

Req.	Met	Evidence	Path
DM1	Y	Section 3.2	`NTMonad/Defs.lean`
DM2	Y	Section 4.4	`ITree/Monad.lean`
DS1	Y	Section 4.4	`ITree/Monad.lean`

(c) NT Monad

Req.	Met	Evidence	Path
AM1	Y	Section 3.2	`AltRepr.lean`
AM2	Y	Section 3.2	`AltRepr.lean`
AM3	Y	Section 4.2	`AltRepr.lean`
AO1	Y	Section 4.2	`AltRepr.lean`

(d) AltRepr Type

Req.	Met	Evidence	Path
FM1	Y	Section 3.6	`M/Free.lean`
FM2	Y	Section 3.6	`M/Free.lean`
FS1	Y	Section 3.6	`M/Free.lean`
FS2	Y	Section 3.6	`M/Free.lean`
FO1	Y	Section 3.6	`M/Free.lean`

(e) Free

Table 4: Requirements and completion

4.1 State-Machine encoding (core)

We will compare the performance of M-type implementations. The implementations I will test are:
1. The $\mathcal{U}$-universe State-Machine M-type,
2. the `SM`-type (Section 3.2.3) which as minimal possible overhead at cost of a universe bump,
3. the progressive approximation encoding from Mathlib,
4. the implementation by Michael Sammler which was started during this project [6].

Notably this final implementation is based on the progressive approximation encoding, but functions are defined using domain theoretic methods. A result of this is the ability to define diverging functions.

I implemented streams for each implementation. Then I measured how long it took to destructure n-layers of the stream of natural numbers using a monotonic clock. This means the x axis of the graph will show the number of destructures $n \times \mathcal{O}(f)$ if an encoding has a complexity f. I repeated this experiment 5 times. For the progressive approximation encoding and Michael Sammler's implementation, I swept $n \in [0, 200]$, and for $\mathcal{U}$-universe State-Machine M-type and SM encodings $n \in [0, 5000]$. I fit polynomials for each of these implementations, then I plot samples, along with the fit. This generates Figure 29.

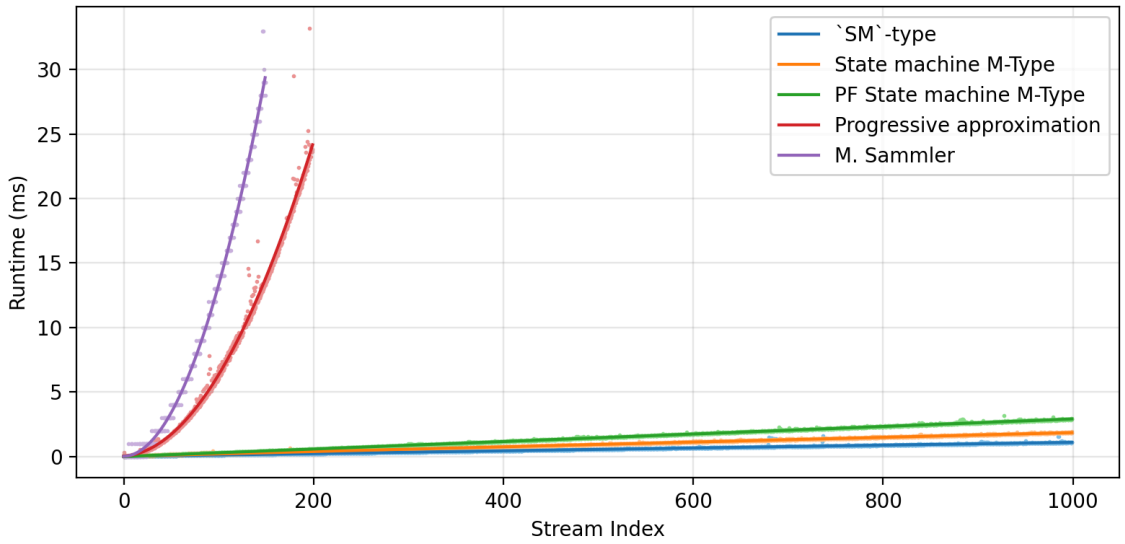


Figure 29: Cumulative performance for stream destructing

In Figure 29 we see the `SM` and $\mathcal{U}$-universe State-Machine M-type are both linear and the implementations build on progressive approximation are quadratic. As this shows $n \times \mathcal{O}(f)$, we can see that destructuring of the state-machine encoding is $\mathcal{O}(1)$, as opposed to the progressive approximation encoding which is $\mathcal{O}(n)$. This is in line with the expected theoretical performance.

In addition to Figure 29, which mainly shows the asymptotics of the different implementations, I also computed a per destructure cost. This was done by dividing the samples by the stream index we are testing (Figure 30). The plot then shows the constant factor of the two implementations.

The difference between `SM` and the $\mathcal{U}$-universe State-Machine M-type comes from the destructor function needing to do a universe lowering. In practice this means the $\mathcal{U}$-universe State-Machine M-type adds two pointer indirections over the `SM` adding a fixed cost at each iteration, compared to the `SM` which calls the destructor function directly.

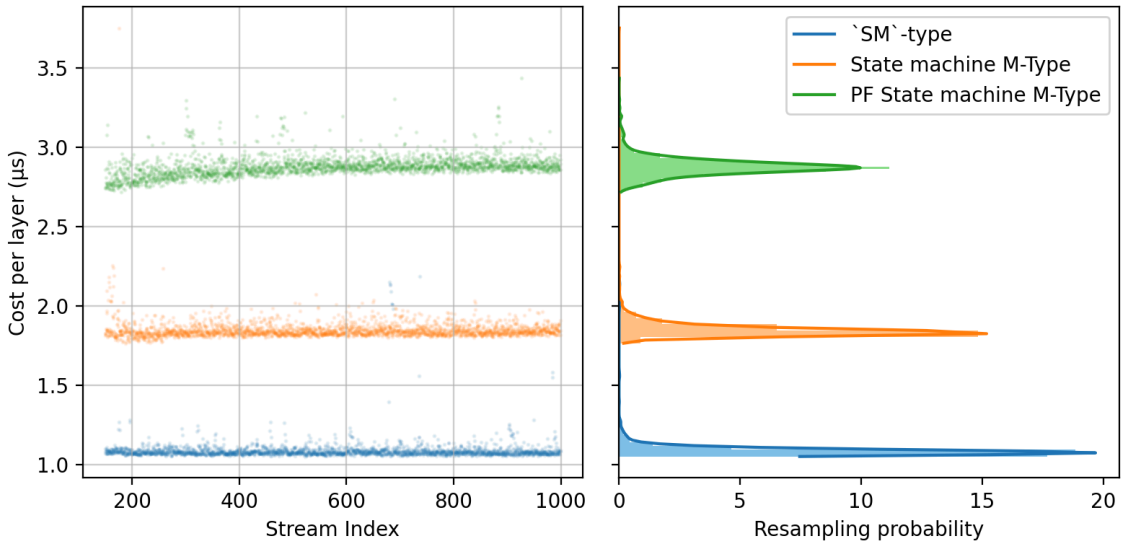


Figure 30: Per-layer performance for stream destructing

All success criteria for the SME type are met.

4.2 AltRepr Type (extension)

Going through the requirements of the AltRepr type, I implemented it (AM1), and added an eliminator (AM2). We now have to assess whether usage is zero cost (AO1).

It is not clear how one would test the performance of the AltRepr type. So rather we can read the generated intermediate representation (LCNF) for the functions we care about. Inspecting the intermediate representation of `mkB` Listing 23b and `destB` Listing 23a, we can see they have become the identity. The eliminator `rec` has also compiled into simply calling the efficient implementation. Additionally, each of these are marked with an `@[inline]` hint, meaning that in compiled code they do not even appear. This also lets us verify that we have the behaviour of the type `B` (AM3).

```
[Compiler.result] size: 0
def AltRepr.destB A B eq a.1 : lcAny :=
  return a.1
```

(a) LCNF `AltRepr.destB`

```
[Compiler.result] size: 0
def AltRepr.mkB A B eq a.1 : lcAny :=
  return a.1
```

(b) LCNF `AltRepr.mkB`

```
[Compiler.result] size: 1
def AltRepr.rec A B eq motive.1 hLog hCheap eqA
eqB v : lcAny :=
  let _x.2 := hCheap v;
  return _x.2
```

(c) LCNF `AltRepr.rec`

Listing 23: LCNF for functions on the AltRepr Type

All success criteria for the AltRepr Type are met.

4.3 Interaction Trees (extension)

For evaluating interaction trees, there are 3 main aspects to evaluate: function coverage, proof coverage, and performance. Since performance is inherited directly from the performance of the M-Type, I will focus on function and proof coverage.

Function coverage can be found in Table 1 and proof coverage Table 2. This is comparing against the ITree paper [2]. In private conversation with `oklch(73%, 0.15, 30deg)`, he informed me of some of the additions to the interaction tree library for Rocq. These include relations such as simulation up to taus (`sutt`), and relation up to taus (`rutt`). For instance, `sutt` is useful for compiler verification in compcert [26]. In C, a non-terminating function is UB and therefore should be able to be related to any other function. This is in line with the fact that the spinning ITree can be simulated by any other ITree. This is as opposed to how the spinning ITree is unique up to weak bisimulation.

Lean now has an ITree library, something multiple groups have requested. For this reason Tobias Grosser has expressed interest in using the current implementation for his project VeIR [27].

During the production of this dissertation, Michael Sammler also started work on an interaction tree library using the progressive approximation encoding and domain theoretic means. A consequence is that his definition of `iter` is not required to be productive.

The domain construction of the coinductive data types allow for very natural function definitions, though unfortunately the definition is still slow. From this regard both implementations have their merits.

All success criteria for interaction trees are met.

4.4 The delay monad (core)

The delay monad is a special case of interaction trees with no visible events. As a consequence of this, I will rather let the user implement the delay monad using interaction trees. As stated above interaction trees form a lawful monad, thereby through the generalization completing the requirements DM2 and DS1.

All success criteria for the delay monad are completed by interaction trees.

4.5 The Free monad (extension)

We will evaluate the Free monad similarly to how we evaluated interaction trees; by comparing function coverage. In addition to this we will compare implementations between functions written using the corecursor directly and functions using the Free monad.

By inspecting the Table 3, we can see we have implemented a futumorphism (FM1) and an injection (FM2). In the code one can also find the cross universe futumorphism (FO1). We also have theorems about flattening, mapping and injection proven in the code, these fill out the `simp`-set for futumorphism. The result of this is that when an end-user uses a futumorphism, all they have to provide is a mapping over constructor lemma, and the rest will automatically solve. The main theorems that make this possible, is the unfolding of futumorphism lemmas. For the case of `futu'`, the statement can be seen in Listing 24 (FS1). Furthermore, I added theorems stating `inject` is an injection, and `flatten` is `inject`'s left-inverse.

```
theorem futu'_flatten
    (f : β → P (α ::: Free P α β))
    (seed : β)
    : (futu' f seed).dest
    = (TypeVec.id ::: flatten ∘ map.{u} (futu' f))
    <$$> f seed := by
  rw [futu'_flatten_mk, M.mk_dest]
```

Listing 24: Futumorphic unfolding lemma

For evaluating the futumorphism, it is best to put practice over promise. For this we can look at some functions written using the current 2 possible styles of writing functions. These are directly using the corecursor and using a futumorphism. I also tried to compare it to Michael Sammler, but the fixpoint turned out to try to compute the entire definition in finite memory, crashing my compiler, this means these functions would not be useful to compute with.

The main use of a futumorphism is moving any recursive component of a corecursor out of the state domain, and into an explicit function. Doing this transformation lets the function be written exactly as one would expect to write a normal inductive function, just inserting corecursive calls as `recall` constructors. Three good examples of this can be seen in Listing 25, Listing 26 and Listing 27.

This fact means that a productive function is exactly a function whose generating function to the futumorphism is terminating.

```
def interlace_const (o : α) (v : SS α)
    : SS α := .corec' body ⟨v, .true⟩
where body : SS α × Bool → _
  | ⟨v, .true⟩ ⇒
    have ⟨hd, tl⟩ := v.dest
    SS.mk hd ⟨tl, .false⟩
  | ⟨v, .false⟩ ⇒ SS.mk o ⟨v, .true⟩

/-- info: [1, 0, 2, 0, 3, 0, 4, 0, 5, 0] -/
#guard_msgs in
#eval! (interlace_const 0 (nats 1)).take 10
```

(a) Corec definition

```
def interlace_const' (o : α)
    : SS α → SS α := futu' body
where body s :=
  have ⟨hd, tl⟩ := s.dest
  SS.mk hd ◁ cont' ◁ SS.mk o ◁ recall tl

/-- info: [1, 0, 2, 0, 3, 0, 4, 0, 5, 0] -/
#guard_msgs in
#eval! (interlace_const' 0 (nats 1)).take 10
```

(b) Futu definition

Listing 25: Interlace constant

```
def stutter (v : SS α)
    : SS α := .corec' body ⟨v, .true⟩
where body : SS α × Bool → _
  | ⟨v, .true⟩ ⇒
    have ⟨hd, _⟩ := v.dest
    SS.mk hd ⟨v, .false⟩
  | ⟨v, .false⟩ ⇒
    have ⟨hd, tl⟩ := v.dest
    SS.mk hd ⟨tl, .true⟩

/-- info: [0, 0, 1, 1, 2, 2, 3, 3, 4, 4] -/
#guard_msgs in
#eval! (stutter (nats 0)).take 10
```

(a) Corec definition

```
def stutter'
    : SS α → SS α := futu' body
where body s :=
  have ⟨hd, tl⟩ := s.dest
  SS.mk hd ◁ cont' ◁ SS.mk hd ◁ recall tl

/-- info: [0, 0, 1, 1, 2, 2, 3, 3, 4, 4] -/
#guard_msgs in
#eval! (stutter' (nats 0)).take 10
```

(b) Futu definition

Listing 26: Stutter constant

```
def RLE_decode
    (s : SS (α × Nat))
    : SS α := .corec' f s.dest where
  f
  | ⟨⟨v, 0  ⟩, r⟩ ⇒ SS.mk v r.dest
  | ⟨⟨v, n+1⟩, r⟩ ⇒ SS.mk v ⟨⟨v, n⟩, r⟩

/-- info: [0, 1, 1, 2, 2, 2, 3, 3, 3, 3] -/
#guard_msgs in
#eval! (RLE_decode ((fun | .fz, a ⇒ Prod.mk a a)
  <$$> nats 0)).take 10
```

(a) Corec definition

```
def RLE_decode'
    : SS (α × Nat) → SS α := Free.futu' f where
  f x :=
    have ⟨⟨v, n⟩, r⟩ := x.dest
    iter v r n
  iter v r
    | 0   ⇒ SS.mk v ◁ .recall r
    | n+1 ⇒ SS.mk v ◁ .cont' ◁ iter v r n

/-- info: [0, 1, 1, 2, 2, 2, 3, 3, 3, 3] -/
#guard_msgs in
#eval! (RLE_decode' ((fun | .fz, a ⇒ Prod.mk a a)
  <$$> nats 0)).take 10
```

(b) Futu definition

Listing 27: Run length decoding

All success criteria for the Free monad are met.

Chapter 5: Conclusions

Coinductive data types are useful not only as a program verification tool, but also in general purpose computation. To this point Lean has only had implementations intended for theoretical reasoning, from work such as J. Avigad, M. Carneiro, and S. Hudon [5] and A. Keizer [4]. There continues to be interest in this topic on the Lean forums (Zulip), and from other researchers with M. Sammler [6]. All prior work is slow, rendering it unusable for general purpose computation.

Lean now has a coinduction library with functions usable in general purpose computation. This means that we can now write, and verify non-terminating programs, and have the complexity we would like from the program. To aid in writing these functions, futumorphisms were implemented. These allow for mixed corecursive-recursive definitions to be written in a more natural way.

On top of this, this dissertation includes a feature rich interaction tree library. This has potential to be used directly in work that requires formally verified compilation or translation. T. Grosser has expressed interest in using this on extending VeIR [27].

I have met all my success criteria, along with universe changing type, an ITree implementation and a futumorphism implementation.

5.1 Future work

On completion of this project, there are now 3 coinduction libraries for Lean. The different abilities of the projects can be seen in Table 5. Each of the libraries now have their own contributions but currently are all incompatible. The goal would be to make one library to unite all of these features. This project would be more work than this dissertation.

	The project	A. Keizer [4]	Michael Sammler
`dest` performance	$\mathcal{O}(1)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$
Type Macro	N	Y[25]	N
Function definitions	`futu`	-	`partial_fixpoint`

Table 5: Comparison of coinductive libraries

5.1.1 Syntax macro for types

A. Keizer [4] puts forward a macro for constructing QPFs [5]. This is a required feature for any coinduction library, for this reason it would be highly desirable to have this. I have designed all the functions on the $\mathcal{U}$-universe State-Machine M-type to be drop in replacements for `M`. This means it should be as simple as updating Alex's library to Lean 4.29, and updating the relevant occurrences.

5.1.2 Derecursifier for corecursive function

A more ambitious goal is finding a way to write a derecursifier for corecursive functions. This would be a macro taking a definition, and transforming a function body into a corecursive function. This would work similarly to `partial_fixpoint` but would need further work. One of the main reasons the futumorphism was implemented, was to serve as a middleground here, but it might even be possible to use it in the derecursifier.

[25] For Lean v4.25

Bibliography

[1] L. M. de Moura, S. Kong, J. Avigad, F. van Doorn, and J. von Raumer, "The Lean Theorem Prover (System Description)," in *Automated Deduction - CADE-25 - 25th International Conference on Automated Deduction, Berlin, Germany, August 1-7, 2015, Proceedings*, 2015, pp. 378–388. doi: 10.1007/978-3-319-21401-6_26.

[2] L.-yao Xia *et al.*, "Interaction trees: representing recursive and impure programs in Coq," *Proc. ACM Program. Lang.*, vol. 4, no. POPL, Dec. 2019, doi: 10.1145/3371119.

[3] The mathlib Community, "The Lean Mathematical Library," in *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, in CPP 2020. New Orleans, LA, USA: ACM, Jan. 2020. doi: 10.1145/3372885.3373824.

[4] A. Keizer, "Implementing a definitional (co)datatype package in Lean 4, based on quotients of polynomial functors," Master's thesis, https://eprints.illc.uva.nl/id/eprint/2239/1/MoL-2023-03.text.pdf, 2023.

[5] J. Avigad, M. Carneiro, and S. Hudon, "Data Types as Quotients of Polynomial Functors," in *10th International Conference on Interactive Theorem Proving (ITP 2019)*, J. Harrison, J. O'Leary, and A. Tolmach, Eds., in Leibniz International Proceedings in Informatics (LIPIcs), vol. 141. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2019, p. 6:1–6:19. doi: 10.4230/LIPIcs.ITP.2019.6.

[6] M. Sammler, "Coinductive." https://github.com/ISTA-PLV/coinductive/tree/2b2ee94fa326e96835e021556ee52fcf6a235ea2, Apr. 2026.

[7] The Univalent Foundations Program, *Homotopy Type Theory: Univalent Foundations of Mathematics*. Institute for Advanced Study: https://homotopytypetheory.org/book, 2013.

[8] S. Awodey, "Cambridge Lectures on Categorical Logic and Type Theory." https://awodey.github.io/cltt/, Mar. 2026.

[9] nLab authors, "type universe." May 2026.

[10] T. Altenkirch, "Computer Aided Formal Reasoning." Accessed: May 11, 2026. [Online]. Available: https://web.archive.org/web/20260511114900/https://people.cs.nott.ac.uk/psztxa/g53cfr/

[11] nLab authors, "polynomial functor." Jan. 2026.

[12] J. Lambek, "A Fixpoint Theorem for complete Categories.," *Mathematische Zeitschrift*, vol. 103, pp. 151–161, 1968, [Online]. Available: http://eudml.org/doc/170906

[13] J. Adámek, "Free algebras and automata realizations in the language of categories," *Commentationes Mathematicae Universitatis Carolinae*, vol. 15, no. 4, pp. 589–602, 1974, [Online]. Available: http://eudml.org/doc/16649

[14] M. P. Fiore, "A Coinduction Principle for Recursive Data Types Based on Bisimulation," *Information and Computation*, vol. 127, no. 2, pp. 186–198, 1996, doi: https://doi.org/10.1006/inco.1996.0058.

[15] Z. Yang and N. Wu, "Fantastic Morphisms and Where to Find Them," in *Mathematics of Program Construction*, E. Komendantskaya, Ed., Cham: Springer International Publishing, 2022, pp. 222–267.

[16] T. Uustalu and V. Vene, "Primitive (Co)Recursion and Course-of-Value (Co)Iteration, Categorically," *Informatica*, vol. 10, p. , Oct. 1999.

[17] R. Hinze, N. Wu, and J. Gibbons, "Unifying structured recursion schemes," in *Proceedings of the 18th ACM SIGPLAN International Conference on Functional Programming*, in ICFP '13. Boston, Massachusetts, USA: Association for Computing Machinery, 2013, pp. 209–220. doi: 10.1145/2500365.2500578.

[18] B. Goodspeed, "Proof Driven Development," *CoRR*, vol. abs/1512.02102, 2015, [Online]. Available: http://arxiv.org/abs/1512.02102

[19] S. O. Bradner, "Key words for use in RFCs to Indicate Requirement Levels." [Online]. Available: https://www.rfc-editor.org/info/rfc2119

[20] J. Kim, Y. Nam, and C.-K. Hur, "Coco: Corecursion with Compositional Heterogeneous Productivity," *Proc. ACM Program. Lang.*, vol. 10, no. POPL, Jan. 2026, doi: 10.1145/3776733.

[21] V. Capretta, "General Recursion via Coinductive Types," *CoRR*, vol. abs/cs/0505037, 2005, [Online]. Available: http://arxiv.org/abs/cs/0505037

[22] nLab authors, "Kleisli category." May 2026.

[23] Y. Zakowski, P. He, C.-K. Hur, and S. Zdancewic, "An equational theory for weak bisimulation via generalized parameterized coinduction," in *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, in CPP 2020. New Orleans, LA, USA: Association for Computing Machinery, 2020, pp. 71–84. doi: 10.1145/3372885.3373813.

[24] D. Pous, "Coinduction All the Way Up," in *Proceedings of the 31st Annual ACM/IEEE Symposium on Logic in Computer Science*, in LICS '16. New York, NY, USA: Association for Computing Machinery, 2016, pp. 307–316. doi: 10.1145/2933575.2934564.

[25] The Agda Team, "Universe Levels — Agda 2.8.0 Documentation." Accessed: May 03, 2026. [Online]. Available: https://agda.readthedocs.io/en/v2.8.0-r3/language/universe-levels.html

[26] X. Leroy, "Formal verification of a realistic compiler," *Commun. ACM*, vol. 52, no. 7, pp. 107–115, Jul. 2009, doi: 10.1145/1538788.1538814.

[27] M. Fehr, T. Grosser, L. Cicolini, and L. Stefanesco, "Coinductive." https://github.com/opencompl/veir/tree/ca1dd404e93e93ac2eb94ab7a36da578bd20b6f1, Apr. 2026.

6 Appendicies

6.A Inductive predicate

An inductive predicate is a least fixed point; a universal quantification over all relations of a given shape. An example of an inductive predicate is evenness. This is defined as a higher-order predicate called the `Shape` in this case, we use this to restrict the relations we are talking about. Then we take the universal over this shape and we get what is shown in Figure 31. It can be shown that this definition is equal to the one generated by the expected Lean code.

```
def IsEven.Shape (P : Nat → Prop) : Prop :=
  P 0 ∧ ∀ n, P n → P (n + 2)
def IsEven (n : Nat) : Prop :=
  ∀ P, IsEven.Shape P → P n
def IsEven.z : IsEven 0 := fun P hP ⇒ hP.left
def IsEven.ss n (h : IsEven n) : IsEven (n+2) :=
  fun P hP ⇒ hP.right n (h P hP)
```

(a)

$$\text{shape}_P \triangleq P0 \wedge \forall n, Pn \to P(n+2)$$
$$\text{Even } (n) \triangleq \forall P, \text{shape}_P \to Pn$$
$$e_0 : \text{Even } 0 \triangleq \lambda P \text{ hP}.\pi_1(\text{hP})$$

(b)

Figure 31: Evenness inductive predicate, from first principles

6.B Coinductive predicate

Given what we saw in the last section and how we have seen the duality for coinductive and inductive data; all we must do is dualize this construction. So we change the universal to an existential, disjuncts to conjuncts, and flip the arrows (each of these swap left and right adjoints). This gives us a proof principal with three components. We now need to define an invariant, we need to prove the invariant satisfies the restriction, and that our value is contained in the invariant.

Proving coinductive predicates is challenging as it requires coming up with the invariant. This is a kind of proof I had no experience in, and it took a while to pick up. An additional difficulty is, if you get the invariant wrong, you have to start completely again and try to come up with something better. This means that for some of my proofs I wrote hundreds of lines of code that have not made it into the dissertation.

Some languages have facilities to make this better, these are collectively known as coinduction-up-to. These are techniques where you can reuse earlier proofs and automatically expand the invariant. For Rocq a popular library for this is Paco[23], a lot of harder coinduction proofs use this technique. Lean does not have this feature, and it is out of scope for the dissertation. In its place I spend a lot of time picking the shape of the Invariant I need.

6.C Benchmarking code

```
import Sme
import Sme.UniM.MultiM

open Sme MvPFunctor

namespace Test

def QStream.Base : MvPFunctor 2 := ⟨
  PUnit,
  fun _ _ ⇒ PUnit
⟩

def QStreamSl α := MvPFunctor.M QStream.Base (fun _ ⇒ α)
def QStreamHp α := Sme.M QStream.Base (fun _ ⇒ α)
def QStreamHp' α := Sme.MM QStream.Base (fun _ ⇒ α)
def QStreamPreM α := PreM QStream.Base (fun _ ⇒ α)
def QStreamSM α := SM QStream.Base (fun _ ⇒ α)

structure QStreamBig.{u} (α : Type u) where
```

```
  corec ::
    {t : Type u}
    (functor : t → QStream.Base !![α, t])
    (obj : t)

def QStreamBig.dest {α} (x : QStreamBig α) : QStream.Base !![QStreamBig α, PLift α] :=
  have ⟨.unit, tl⟩ := x.functor x.obj
  ⟨.unit, fun
    | .fz, .unit ⇒ .up ◁ tl (.fs .fz) .unit
    | .fs .fz, .unit ⇒ .corec x.functor ◁ tl .fz .unit⟩

/- set_option trace.compiler.ir.result true in -/
section

def numsSl : QStreamSl Nat :=
  .corec _ (fun n ⇒ ⟨.unit, fun | .fz, .unit ⇒ n.succ | .fs .fz, .unit ⇒ n⟩) Nat.zero

/- def numsHp : QStreamHp Nat := -/
/-   .corec' (fun n ⇒ ⟨.unit, fun | .fz, .unit ⇒ n.succ | .fs .fz, .unit ⇒ n⟩) Nat.zero -/

def numsHp : QStreamHp.{0} Nat :=
  Sme.M.corec.{0,0} (fun n ⇒ ⟨
    .up .unit,
    fun | .fz, .up .unit ⇒ .up ◁ .up ◁ n.down.succ | .fs .fz, .up .unit ⇒ n⟩)
    ◁ ULift.up Nat.zero

def numsHp' : QStreamHp'.{0} Nat :=
  Sme.MM.corec.{0,0} (fun n ⇒ ⟨
    .up .unit,
    fun | .fz, .up .unit ⇒ .up ◁ .up ◁ n.down.succ | .fs .fz, .up .unit ⇒ n⟩)
    ◁ ULift.up Nat.zero

def numsPreM : QStreamPreM.{0} Nat :=
  .corec (fun n ⇒ ⟨
    .up .unit,
    fun | .fz, .up .unit ⇒ .up ◁ .up ◁ n.down.succ | .fs .fz, .up .unit ⇒ n⟩) ◁ .up Nat.zero

def numsSM : QStreamSM.{0} Nat :=
  .corec (fun n ⇒ ⟨
    .up .unit,
    fun | .fz, .up .unit ⇒ .up ◁ .up ◁ n.down.succ | .fs .fz, .up .unit ⇒ n⟩) ◁ .up Nat.zero

def numsBig : QStreamBig Nat :=
  QStreamBig.corec (fun n ⇒ ⟨.unit, fun | .fz, .unit ⇒ n.succ | .fs .fz, .unit ⇒ n⟩) 0

def QStreamBig.getNth (x : QStreamBig Nat) : Nat → Nat
  | 0 ⇒ match x.dest with
    | ⟨.unit, v⟩ ⇒ (v .fz .unit).down
  | n+1 ⇒ match x.dest with
    | ⟨.unit, v⟩ ⇒ QStreamBig.getNth (v (.fs .fz) .unit) n

def QStreamSl.getNth (x : QStreamSl Nat) : Nat → Nat
  | 0 ⇒ match x.dest with
    | ⟨.unit, v⟩ ⇒ v (.fs .fz) .unit
  | n+1 ⇒ match x.dest with
    | ⟨.unit, v⟩ ⇒ QStreamSl.getNth (v .fz .unit) n

def QStreamPreM.getNth (x : QStreamPreM Nat) : Nat → Nat
  | 0 ⇒ match x.dest with
    | ⟨.up .unit, v⟩ ⇒ (v (.fs .fz) ◁ .up .unit).down
  | n+1 ⇒ match x.dest with
    | ⟨.up .unit, v⟩ ⇒ QStreamPreM.getNth (v .fz ◁ .up .unit) n

def QStreamSM.getNth (x : QStreamSM Nat) : Nat → Nat
  | 0 ⇒ match x.dest with
    | ⟨.up .unit, v⟩ ⇒ (v (.fs .fz) ◁ .up .unit).down
  | n+1 ⇒ match x.dest with
    | ⟨.up .unit, v⟩ ⇒ QStreamSM.getNth (v .fz ◁ .up .unit) n

def QStreamHp.getNth (x : QStreamHp Nat) : Nat → Nat
  | 0 ⇒ match x.dest with
    | ⟨.unit, v⟩ ⇒ v (.fs .fz) .unit
  | n+1 ⇒ match x.dest with
    | ⟨.unit, v⟩ ⇒ QStreamHp.getNth (v .fz .unit) n

def QStreamHp'.getNth (x : QStreamHp' Nat) : Nat → Nat
  | 0 ⇒ match x.dest with
    | ⟨.unit, v⟩ ⇒ v (.fs .fz) .unit
  | n+1 ⇒ match x.dest with
    | ⟨.unit, v⟩ ⇒ QStreamHp'.getNth (v .fz .unit) n

end

def time (f : Unit → IO Unit) : IO Nat := do
  let pre ← IO.monoNanosNow
  f ()
  let ante ← IO.monoNanosNow
  return (ante - pre)

def runs {A} (s : Nat) (io : IO (List A)) : IO (List (List A)) :=
```

```
  (List.replicate s io).mapM id

def runTests : IO Unit := do
  let testHp   := fun n _ ⇒ do if (QStreamHp.getNth numsHp n) ≠ n then println! "NEQ!";
  let testHp'  := fun n _ ⇒ do if (QStreamHp'.getNth numsHp' n) ≠ n then println! "NEQ!";
  let testSl   := fun n _ ⇒ do if (QStreamSl.getNth numsSl n) ≠ n then println! "NEQ!";
  let testSM   := fun n _ ⇒ do if (QStreamSM.getNth.{0} numsSM n) ≠ n then println! "NEQ!";
  let testPreM := fun n _ ⇒ do if (QStreamPreM.getNth.{0} numsPreM n) ≠ n then println! "NEQ!";
  let testBig  := fun n _ ⇒ do if (numsBig.getNth n) ≠ n then println! "NEQ!";

  let s := 3

  let n := 200
  let sl := (List.range n).map (time ∘ testSl)

  let n := 1000
  let hp   := (List.range n).map (time ∘ testHp)
  let hp'  := (List.range n).map (time ∘ testHp')
  let big  := (List.range n).map (time ∘ testBig)
  let preM := (List.range n).map (time ∘ testPreM)
  let sm   := (List.range n).map (time ∘ testSM)

  /- IO.print "slRuns = " -/
  /- let res ← runs s ◁ sl.mapM id -/
  /- println! res -/

  IO.print "hpRuns = "
  let res ← runs s ◁ hp.mapM id
  println! res

  IO.print "hpARuns = "
  let res ← runs s ◁ hp'.mapM id
  println! res

  /- IO.print "bigRuns = " -/
  /- let res ← runs ◁ big.mapM id -/
  /- println! res -/

  IO.print "smRuns = "
  let res ← runs s ◁ sm.mapM id
  println! res

  IO.print "preMRuns = "
  let res ← runs s ◁ preM.mapM id
  println! res

  return ()

/- #eval! runTests -/
/- #eval runTestsHp -/

end Test
import ITree.Definition
import ITree.Effects.Log

open ITree ITree Effects

def test (n : Nat) : ITree (logE Nat) Empty :=
  .vis n fun _ ⇒ test (n + 1)
partial_fixpoint

def dest (h : ITree (logE Nat) Empty) : Nat → Nat
  | 0 ⇒ match unfold h with
    | .ret x ⇒ x.elim
    | .tau t ⇒ 0
    | .vis e _ ⇒ e
  | n+1 ⇒ match unfold h with
    | .ret x ⇒ x.elim
    | .tau t ⇒ dest t n
    | .vis _ t ⇒ dest (t .unit) (n)

#time #eval! dest (test 0) 10
/- #time #eval! dest (test 0) 1000 -/
/- #time #eval! dest (test 0) 1500 -/
/- #time #eval! dest (test 0) 1750 -/

def time (f : Unit → IO Unit) : IO Nat := do
  let pre ← IO.monoMsNow
  f ()
  let ante ← IO.monoMsNow
  return ante - pre

def out : IO PUnit := do
  let t  := fun n _ ⇒ do if (dest (test 0) n) ≠ n then println! "NEQ!";

  let n := 400
  let s := 3

  let arr := (List.range n).map (time ∘ t)
```

```
  let runs := fun io ⇒ (List.replicate s io).mapM id
  let res ← runs ◁ arr.mapM id
  println! "msRuns = "
  println! res

/- #eval out -/
```

6.D Unsoundness of old shrink definition

All credit goes to Aaron Liu from the Mathlib community.

```
import Mathlib.Logic.Small.Defs
import Mathlib.Data.Setoid.Basic

def Nat' := Quotient (Setoid.ker fun x : Nat ↦ x - 1)
@[simp] theorem mk_zero : (⟦0⟧ : Nat') = ⟦1⟧ := Quotient.sound rfl

def equivNat : Nat' ≃ Nat where
  toFun x := x.lift _ fun _ _ ↦ id
  invFun x := ⟦x + 1⟧
  left_inv x := x.inductionOn fun x ↦ x.recOn (by simp) (by simp)
  right_inv x := by simp

theorem Shrink.eq_of_equiv {α : Type u} {β : Type v} [Small.{w} α] [Small.{w} β]
    (equiv : α ≃ β) : Shrink.{w} α = Shrink.{w} β := by
  unfold Shrink
  have hu (x : Type w) : Nonempty (α ≃ x) = Nonempty (β ≃ x) :=
    propext ⟨Nonempty.map equiv.symm.trans, Nonempty.map equiv.trans⟩
  simp [hu]

def castNat : Shrink.{0} Nat' → Shrink.{0} Nat := cast (Shrink.eq_of_equiv equivNat)

theorem contradiction : False := by
  have : (equivShrink _).symm (castNat (equivShrink _ ⟦0⟧)) ≠
      (equivShrink _).symm (castNat (equivShrink _ ⟦1⟧)) := by native_decide
  simpa

/- 'contradiction' depends on axioms:
[propext, Classical.choice, Lean.ofReduceBool, Lean.trustCompiler, Quot.sound] -/
#print axioms contradiction
```

6.E Project import graph

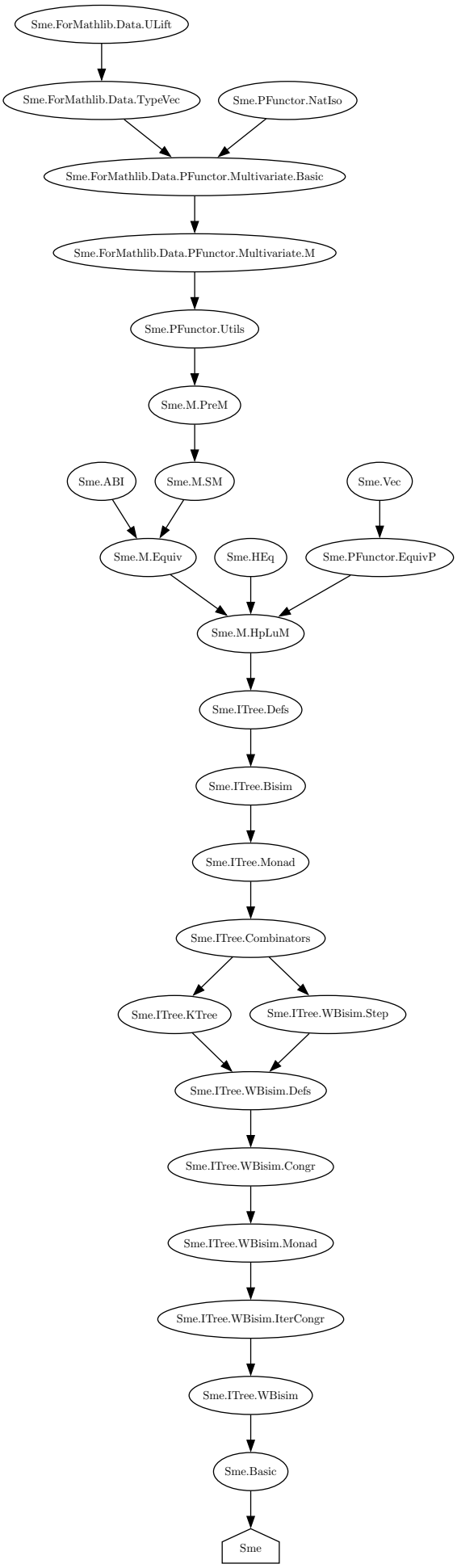


Figure 32: Import graph of the lean component of the project as of 2026-04-04

7 Proposal

See proposal overleaf

Efficient coinductives through state-machine corecursors

Supervisor: Alex Keizer

Marking supervisor: Jamie Vicary

Director of studies: Russell Moore

Word count: 993

1 Work to be undertaken

Previous work has shown how coinductive types can be encoded in Lean as quotients of polynomial functors (QPFs)[1]. This is done through progressive approximation (Section 4.1.2). For example accessing the n index of a stream takes $\mathcal{O}(n^2)$ time. This becomes an issue when using Lean as a general purpose programming language. An alternative approach is using a state-machine based encoding (Section 4.1.1) of cofixed points. With these we get nice computational behaviour of the compiled code. In Figure 1 we can see in blue the performance from the current implementation, versus in orange the performance of a prototype implementation the state-machine based version.

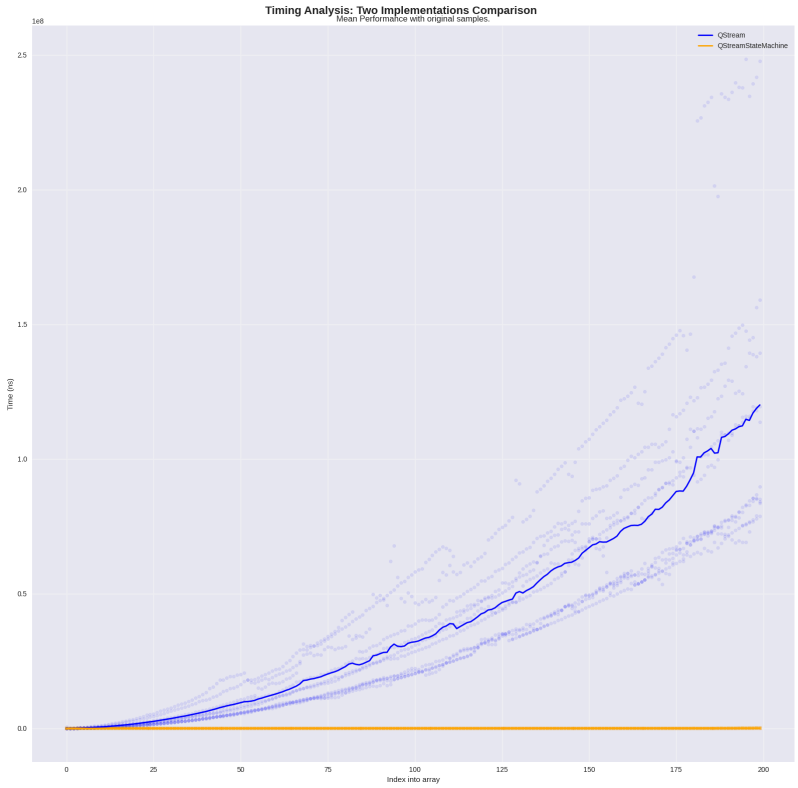


Figure 1: Graph plotting current performance v possible gains

2 Core project

This project will be implementing the state-machine encoding (Section 4.1.1) of coinductive types, and formalising the equivalence between these two embedding (Section 4.3). I will start with the special example of equivalence between `Stream`s in the two representations, then building up to the general cofix structure.

After this I will be implementing a coinductive monad allowing us to encode non-termination as an effect using the state-machine representation. This will be an instance of the two representations.

3 Starting point

I have worked with QPFs on the meta-programming side for an internship between my Part Ia and Part Ib. During this I learnt of the basics of polynomial functors, and `TypeVec`s. This means I am aware of what the underlying structures are when it comes to the raw implementation. I have also done a feasibility assessment of the project by seeing how the current polynomials respond to universe levels. This lead to me making 2 pull requests (#28095, #28279) to mathlib on `TypeVec` in preparation for my project. I also tried to merge (#28112) an implementation of commutable Shrink to mathlib that later got reverted when it was found to be inconsistent.

There are more minor refactoring pull requests towards the mathlib repository which don't change any behaviour but in general all of these can be *found on this link*. I Include these for completeness and transparency.

4 Substance

Section 4.1.1, Section 4.3 and Section 4.4 are all discuss structures that will be implemented throughout the project. Section 4.1.2 and Section 4.2 both are implemented in Avigard et al [1].

4.1 The M type

The M type is the name given to the terminal coalgebra of polynomial functors; the possibly infinitely deep trees. This means the implementations must have both: A a corecursor $\text{corec} : \{\alpha : \text{Type}\} \to (f : \alpha \to F\alpha) \to \alpha \to MF$, and a destructurer $\text{dest} : \text{M}F \to F(MF)$. We have two encodings of the M type.

4.1.1 State-machine encoding

The state-machine encoding is the naïve way to implement the terminal coalgebra. Given some polynomial functor F, the state-machine encoding is given by: some type $\alpha : \text{Type}$, a function $f : \alpha \to F\alpha$, and some witness $a : \alpha$. Then you quotient over bisimilarity, thereby only allowing direct usage of $\text{dest} : \text{Sme}\ F \to F(\text{Sme}\ F)$. The baseline implementation of this should be straight-forward.

4.1.2 Progressive approximation encoding

The other way is to generate them by progressive approximation where earlier trees must "agree" with the later ones. Agreement is given by them being the same up to the previous depth as seen in Figure 2.

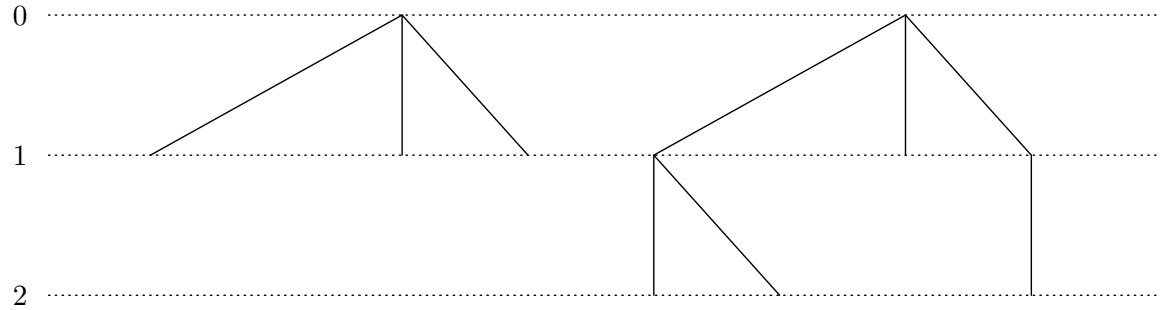


Figure 2: Agreement of two trees of height 1 and 2

4.2 The `Cofix` type

`Cofix` is the terminal coalgebra in QPFs, These are simply a quotient over M types lifting the quotient from the source QPF.

4.3 The equivalence

The equivalence between the current implementation of `Cofix` and M and the state-machine representation is the core of the project. The functions in both directions are given by $f : \text{corec}_{\text{Pae}}\text{dest}_{\text{Sme}}$ and $f^{-1} : \text{corec}_{\text{Sme}}\text{dest}_{\text{Pae}}$. The difficulty comes from proving $f \circ f^{-1} = \mathbb{1}$ and $f^{-1} \circ f = \mathbb{1}$. The proof of these equalities will be proven using bisimilarity, and other parts of the theory currently established for M types. Simpler versions of this equivalence also exist for the simpler representation.

4.4 The non-termination monad

The non-termination monad is a simple example of a coinductive. This will be useful for testing the state-machine representation's performance. The structure of the monad as a coinductive has two constructors as seen in this psudocode:

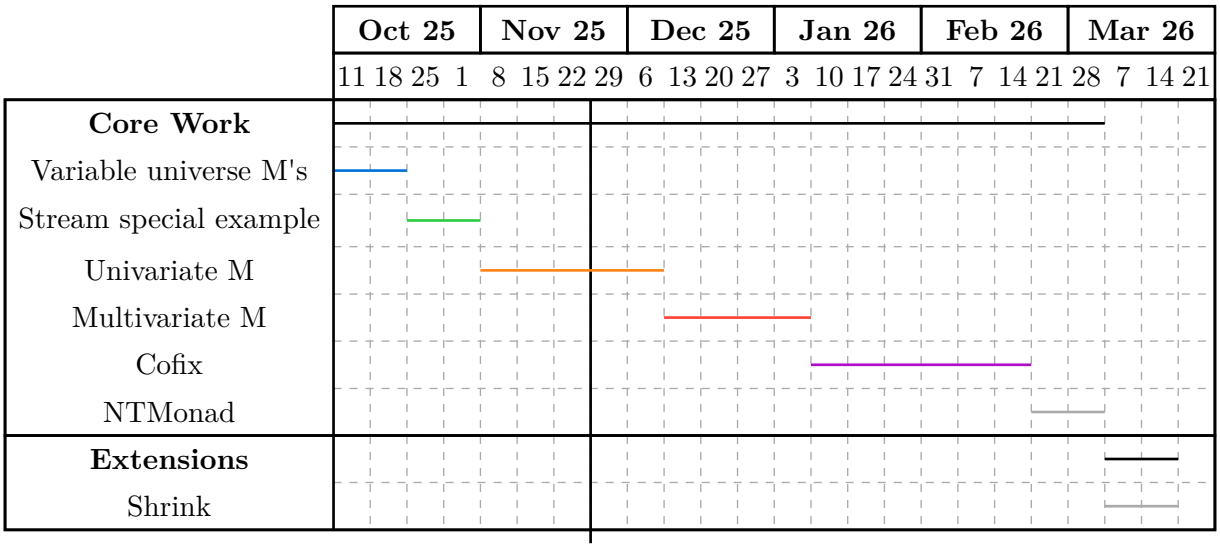
```
coinductive NTMonad (A : Type)
  | val : A -> NTMonad A
  | tau : NTMonad A -> NTMonad A
```

5 Evaluation

This projects success can be broken down into a few disjoint criteria.

1. If the equivalence (Section 4.3) has been proven,
2. how close to the theorised performance we get with the state-machine encoding,
3. and the ability to construct the example structures.

6 Core timeline



The plan for work would be divided into a few different stages.

6.1 Variable universe `M`s (2025-10-11 2w 2025-10-24)

To begin with I have to make the corecursor universe polymorphic. As stated in Section 3 I have most of this code written. I will attempt to get this merged into mathlib, but if this review process takes too long I will be working on my own branch.

6.2 `Stream` special example (2025-10-25 2w 2025-11-7)

The first step would be implementing `Stream` under the two representations. Then I will familiarise myself with using bisimilarity to formalise the equivalence. This is a first step to just understand how these features work.

6.3 Univariate `M` (2025-11-8 2w 2025-11-21 + (CAT exam) + 2025-12-05 3w 2025-12-23)

Implementing the univariate `M` type is the natural next step from this. This will be the natural next step in difficulty. The work should lay the groundwork for the next proofs.

6.4 Multivariate `M` (2025-12-26 4w 2026-01-23)

The next step will be generalising the univariate implementation, the same structure of the proof should be the same for the multivariate case. The main difficulty here comes from working with `TypeVec`s. These are quite difficult to reason about.

6.5 `Cofix` (2026-01-24 6w 2026-03-06)

Finally, the `Cofix` type has to be implemented. This will be the most challenging part as I will have to work with quotients.

6.6 Implementing the NTMonad (2026-03-07 2w 2026-03-20)

This will be a demonstration of all of the previous work.

7 Extensions

7.1 Shrinking the representations (2026-03-21 2w 2026-04-04)

This involves finding a sound way allowing the current theory to use the efficient representation. The soundness of this will be difficult to reason about as it requires working with the Lean code generator. Therefore I leave this as an extension.

8 Resources

Access to the restricted side of the lab is needed for working with the greater team.

Bibliography

[1] J. Avigad, M. Carneiro, and S. Hudon, "Data Types as Quotients of Polynomial Functors," in *10th International Conference on Interactive Theorem Proving (ITP 2019)*, J. Harrison, J. O'Leary, and A. Tolmach, Eds., in Leibniz International Proceedings in Informatics (LIPIcs), vol. 141. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2019, pp. 6:1–6:19. doi: 10.4230/LIPIcs.ITP.2019.6.